Wszystkie zapytania SQL projektu PostgreSQL vs Microsoft SQL Server

Fraud Detection & Customer Risk Analytics · każde zapytanie ze skryptów 00–05, jego wersja dla SSMS, zaznaczone różnice i wyjaśnienie każdej funkcji

Jak czytać ten dokument

Każde zapytanie ma ten sam układ: **co robi**, **wynik** na Twoich danych, kod **PostgreSQL** (dokładnie taki jak w skrypcie), ten sam kod w wersji **SQL Server (SSMS)**, tabela różnic i lista wszystkich użytych funkcji z wyjaśnieniem.

`tak zaznaczony fragment` w kodzie PostgreSQL to miejsce, które w SQL Server wygląda inaczej

`tak zaznaczony fragment` w kodzie SQL Server to jego odpowiednik (albo coś, czego w PostgreSQL nie trzeba)

Ramka **niebieska** = PostgreSQL (Twój projekt), ramka **czerwona** = SQL Server. Komentarze w kodzie są po angielsku, jak w skryptach; wyjaśnienia są po polsku.

Ważne: kod PostgreSQL jest skopiowany ze skryptów i automatycznie sprawdzony co do znaku. Wersje SQL Server są przetłumaczone ręcznie i **nie były uruchamiane** na SQL Server (w tym projekcie nie mamy takiej bazy). Zakładają SQL Server 2017 lub nowszy; miejsca wymagające wersji 2022 są opisane w komentarzach. Skrypty projektu nie zostały zmienione.

Najważniejsze różnice w jednym miejscu

PostgreSQL	SQL Server (T-SQL)	Gdzie w projekcie
`BOOLEAN, true / false, warunek: is_fraud`	`BIT, 1 / 0, warunek: is_fraud = 1`	wszędzie
`COUNT(*) FILTER (WHERE is_fraud)`	`SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END)`	prawie każde zapytanie
`AVG(is_fraud::int)`	`AVG(CAST(is_fraud AS float))`	fraud rate
`x::date, x::numeric, x::text`	`CAST(x AS date / numeric / varchar)`	wszędzie
`JOIN t USING (customer_id)`	`JOIN t ON t.customer_id = x.customer_id`	łączenie tabel
`GROUP BY 1`	`GROUP BY <całe wyrażenie> lub CROSS APPLY + alias`	grupowanie po CASE / dacie
`LIMIT 20 (na końcu)`	`SELECT TOP 20 (na początku)`	rankingi
`TIMESTAMP`	`DATETIME2 (TIMESTAMP w T-SQL to coś innego!)`	tworzenie tabel
`DATE_TRUNC('month', ts)`	`DATEFROMPARTS(YEAR(ts), MONTH(ts), 1) / DATETRUNC (2022)`	trendy
`EXTRACT(HOUR FROM ts)`	`DATEPART(hour, ts)`	pora dnia, sygnał nocny
`ts + INTERVAL '24 hours'`	`DATEADD(hour, 24, ts)`	sygnały, okno 30 dni
`RANGE BETWEEN INTERVAL '10 minutes' PRECEDING`	`brak: OUTER APPLY z warunkiem na czas`	sygnał 5+ tx w 10 min
`CROSS JOIN LATERAL (VALUES ...)`	`CROSS APPLY (VALUES ...)`	unpivot sygnałów i wersji
`PERCENTILE_CONT ... (agregat)`	`PERCENTILE_CONT ... OVER (...) (tylko okno)`	mediana, percentyl 99,9
`BOOL_OR(x)`	`MAX(CAST(x AS int))`	czy klient miał fraud
`CREATE TABLE x AS SELECT`	`SELECT ... INTO x FROM`	fact_transaction_risk
`UPDATE t r SET ... FROM (...) c WHERE ...`	`UPDATE r SET ... FROM t r JOIN (...) c ON ...`	wynik klienta
`ADD COLUMN IF NOT EXISTS`	`IF COL_LENGTH(...) IS NULL ALTER TABLE ... ADD + GO`	kolumny v2
`(a > b) AS kolumna`	`CASE WHEN a > b THEN 1 ELSE 0 END`	sygnały w 03
`AVG(int) daje ułamek`	`AVG(int) obcina ułamek: AVG(1.0 * x)`	średnie z liczników

Spis zapytań

Plik	Nr	Zapytanie
00_create_tables.sql	1	Tabele wymiarów: klienci, sprzedawcy, urządzenia, kursy walut
00_create_tables.sql	2	Tabele faktów: transakcje i ryzyko klienta
00_create_tables.sql	3	Kontrola: lista utworzonych tabel
01_data_quality.sql	1	17 kontroli jakości danych w jednej tabeli
01_data_quality.sql	2	Outliery: transakcje 10× wyższe niż mediana klienta
02_sql_analysis/01_fraud_rate.sql	1	Ogólny fraud rate (liczbowo i kwotowo)
02_sql_analysis/01_fraud_rate.sql	2	Fraud rate według kategorii sprzedawcy
02_sql_analysis/01_fraud_rate.sql	3	Fraud rate według kraju transakcji
02_sql_analysis/01_fraud_rate.sql	4	Transakcja za granicą a ryzyko
02_sql_analysis/02_fraud_trend.sql	1	Trend miesięczny
02_sql_analysis/02_fraud_trend.sql	2	Trend tygodniowy (tydzień od poniedziałku)
02_sql_analysis/02_fraud_trend.sql	3	Trend dzienny + średnia krocząca z 7 dni
02_sql_analysis/02_fraud_trend.sql	4	Pora dnia: o której godzinie najwięcej fraudu?
02_sql_analysis/03_customer_behaviour.sql	1	Top 20 klientów według wartości transakcji
02_sql_analysis/03_customer_behaviour.sql	2	Top 20 klientów według liczby transakcji
02_sql_analysis/03_customer_behaviour.sql	3	Średnia transakcja: klienci z fraudem vs bez fraudu
02_sql_analysis/03_customer_behaviour.sql	4	Liczba urządzeń klienta a ryzyko
02_sql_analysis/04_window_functions.sql	1	ROW_NUMBER i LAG: historia jednego klienta krok po kroku
02_sql_analysis/04_window_functions.sql	2	Sygnały ryzyka dla każdej transakcji i ich skuteczność
03_risk_scoring.sql	1	Część 1: scoring v1 każdej transakcji (tabela fact_transaction_risk)
03_risk_scoring.sql	2	Część 2: wynik ryzyka klienta (UPDATE fact_customer_risk)
03_risk_scoring.sql	3	Kontrola 3a: transakcje według poziomu ryzyka
03_risk_scoring.sql	4	Kontrola 3b: klienci według poziomu ryzyka
04_false_positives.sql	1	Macierz pomyłek dla progu 60
04_false_positives.sql	2	Tabela progów 30 / 50 / 60 / 70 / 80
04_false_positives.sql	3	Które reguły robią fałszywe alarmy? (próg 60)
04_false_positives.sql	4	Najczęstsze kombinacje sygnałów w alarmach (próg 60)
05_risk_score_v2.sql	1	Część 1: wynik v2 każdej transakcji (nowe kolumny)
05_risk_score_v2.sql	2	Część 2: wynik v2 klienta (nowe kolumny w fact_customer_risk)
05_risk_score_v2.sql	3	Porównanie 3a: progi, v1 vs v2 obok siebie
05_risk_score_v2.sql	4	Porównanie 3b: transakcje według poziomu, v1 vs v2
05_risk_score_v2.sql	5	Porównanie 3c: klienci według poziomu, v1 vs v2

00_create_tables.sql

1. Tabele wymiarów: klienci, sprzedawcy, urządzenia, kursy walut

Co robi: Tworzy cztery tabele wymiarów (dim_), czyli „słowniki” opisujące transakcję: kto płacił, gdzie, z jakiego urządzenia i po jakim kursie. Kolejność ma znaczenie: dim_device wskazuje na dim_customer, więc klienci muszą powstać pierwsi.

Wynik na Twoich danych: Cztery puste tabele. Dane ładuje później skrypt Python 06_load_to_postgres.py.

PostgreSQL (Twój skrypt)

```
-- Customers
CREATE TABLE dim_customer (
    customer_id      INT PRIMARY KEY,
    country          CHAR(2)      NOT NULL,          -- ISO country code, e.g. 'PL'
    age              SMALLINT     CHECK (age BETWEEN 18 AND 100),
    signup_date      DATE         NOT NULL,
    plan             VARCHAR(20),                    -- e.g. standard / plus / premium / metal
    account_age_days INT
);

-- Merchants (shops, services)
CREATE TABLE dim_merchant (
    merchant_id            INT PRIMARY KEY,
    merchant_category      VARCHAR(50) NOT NULL,     -- e.g. online_shopping, travel, crypto
    country                CHAR(2),
    merchant_risk_category VARCHAR(10)               -- low / medium / high
);

-- Customer devices
CREATE TABLE dim_device (
    device_id      INT PRIMARY KEY,
    customer_id    INT NOT NULL REFERENCES dim_customer (customer_id),
    device_country CHAR(2),
    first_seen     TIMESTAMP,
    last_seen      TIMESTAMP
);

-- Exchange rates for converting amounts to EUR
CREATE TABLE dim_fx_rate (
    rate_date DATE,
    currency  CHAR(3),
    eur_rate  NUMERIC(12,6) NOT NULL,
    PRIMARY KEY (rate_date, currency)
);
```

Microsoft SQL Server (SSMS)

```
-- Customers
CREATE TABLE dim_customer (
    customer_id      INT PRIMARY KEY,
    country          CHAR(2)      NOT NULL,
    age              SMALLINT     CHECK (age BETWEEN 18 AND 100),
    signup_date      DATE         NOT NULL,
    [plan]           VARCHAR(20),          -- PLAN is a reserved word in T-SQL
    account_age_days INT
);

-- Merchants (shops, services)
CREATE TABLE dim_merchant (
    merchant_id            INT PRIMARY KEY,
    merchant_category      VARCHAR(50) NOT NULL,
    country                CHAR(2),
    merchant_risk_category VARCHAR(10)
);

-- Customer devices
CREATE TABLE dim_device (
    device_id      INT PRIMARY KEY,
    customer_id    INT NOT NULL REFERENCES dim_customer (customer_id),
    device_country CHAR(2),
    first_seen     DATETIME2,
    last_seen      DATETIME2
);

-- Exchange rates for converting amounts to EUR
CREATE TABLE dim_fx_rate (
    rate_date DATE NOT NULL,
    currency  CHAR(3) NOT NULL,
    eur_rate  NUMERIC(12,6) NOT NULL,
    PRIMARY KEY (rate_date, currency)
);
```

```
);
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`TIMESTAMP`	`DATETIME2`	W SQL Server TIMESTAMP to nie data, tylko licznik wersji wiersza. Data z godziną to DATETIME2.
`plan`	`[plan]`	PLAN jest słowem zarezerwowanym w T-SQL, więc nazwę kolumny bierzemy w nawiasy kwadratowe (także później w zapytaniach: c.[plan]).
`rate_date DATE`	`rate_date DATE NOT NULL`	PostgreSQL sam dopisuje NOT NULL kolumnom klucza głównego. SQL Server też to robi przy CREATE TABLE, ale jawne NOT NULL jest czytelniejsze i bezpieczne.

Funkcje i metody użyte w tym zapytaniu

- **CREATE TABLE**: Tworzy nową, pustą tabelę: nazwy kolumn, ich typy i reguły. Działa tak samo w obu systemach.
- **Typy danych**: INT: liczba całkowita; SMALLINT: mała liczba całkowita; BIGINT: duża liczba całkowita; CHAR(n): tekst o stałej długości; VARCHAR(n): tekst do n znaków; DATE: sama data; NUMERIC(p,s): liczba dokładna z s miejscami po przecinku (dobra do pieniędzy). Te typy są takie same w obu systemach.
- **PRIMARY KEY (klucz główny)**: Kolumna (lub zestaw kolumn), która jednoznacznie identyfikuje wiersz. Nie może się powtarzać ani być pusta. Baza sama zakłada na niej indeks, więc wyszukiwanie po kluczu jest szybkie.
- **PRIMARY KEY (a, b)**: Klucz złożony z dwóch kolumn: unikalna musi być dopiero para wartości (np. data + waluta), a nie każda kolumna osobno.
- **REFERENCES (klucz obcy, FOREIGN KEY)**: Kolumna wskazuje na klucz główny innej tabeli. Baza nie pozwoli wpisać wartości, której nie ma w tabeli nadrzędnej (np. transakcji klienta, który nie istnieje). Składnia identyczna w obu systemach.
- **CHECK (warunek)**: Reguła sprawdzana przy każdym zapisie, np. wiek od 18 do 100. Wiersz, który jej nie spełnia, zostanie odrzucony.
- **NOT NULL**: Kolumna nie może być pusta (NULL = brak wartości).
- **TIMESTAMP vs DATETIME2**: W PostgreSQL TIMESTAMP to data z godziną. Uwaga, pułapka: w SQL Server TIMESTAMP oznacza coś zupełnie innego (licznik wersji wiersza, synonim ROWVERSION). Odpowiednikiem jest DATETIME2.

2. Tabele faktów: transakcje i ryzyko klienta

Co robi: Tworzy dwie tabele faktów: fact_transaction (każda płatność, z etykietą is_fraud) i fact_customer_risk (jeden wiersz na klienta: historia i później wynik ryzyka). Klucze obce pilnują, żeby transakcja zawsze wskazywała na istniejącego klienta, sprzedawcę i urządzenie.

Wynik na Twoich danych: Dwie puste tabele: serce schematu gwiazdy.

PostgreSQL (Twój skrypt)

```
-- Transactions (fact table)
CREATE TABLE fact_transaction (
    transaction_id BIGINT PRIMARY KEY,
    customer_id    INT           NOT NULL REFERENCES dim_customer (customer_id),
    tx_datetime    TIMESTAMP     NOT NULL,
    amount         NUMERIC(12,2) NOT NULL,
    currency       CHAR(3)       NOT NULL,
    merchant_id    INT           REFERENCES dim_merchant (merchant_id),
    device_id      INT           REFERENCES dim_device (device_id),
    country        CHAR(2),
    is_fraud       BOOLEAN       NOT NULL DEFAULT false   -- T-SQL: BIT NOT NULL DEFAULT 0
);

-- Customer risk: previous_fraud and chargebacks will come from the data,
-- risk_score and risk_level will be calculated later in the scoring step.
CREATE TABLE fact_customer_risk (
    customer_id    INT PRIMARY KEY REFERENCES dim_customer (customer_id),
    previous_fraud BOOLEAN DEFAULT false,
    chargebacks    INT     DEFAULT 0,
    risk_score     NUMERIC(5,2),
    risk_level     VARCHAR(10)
);
```

Microsoft SQL Server (SSMS)

```
-- Transactions (fact table)
CREATE TABLE fact_transaction (
    transaction_id BIGINT PRIMARY KEY,
```

```
    customer_id    INT           NOT NULL REFERENCES dim_customer (customer_id),
    tx_datetime    DATETIME2     NOT NULL,
    amount         NUMERIC(12,2) NOT NULL,
    currency       CHAR(3)       NOT NULL,
    merchant_id    INT           REFERENCES dim_merchant (merchant_id),
    device_id      INT           REFERENCES dim_device (device_id),
    country        CHAR(2),
    is_fraud       BIT           NOT NULL DEFAULT 0
);

-- Customer risk
CREATE TABLE fact_customer_risk (
    customer_id    INT PRIMARY KEY REFERENCES dim_customer (customer_id),
    previous_fraud BIT DEFAULT 0,
    chargebacks    INT DEFAULT 0,
    risk_score     NUMERIC(5,2),
    risk_level     VARCHAR(10)
);
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`BOOLEAN DEFAULT false`	`BIT DEFAULT 0`	SQL Server nie ma typu logicznego; BIT przechowuje 1 (prawda) lub 0 (fałsz).
`TIMESTAMP`	`DATETIME2`	Jak wyżej: w T-SQL TIMESTAMP nie jest datą.

Funkcje i metody użyte w tym zapytaniu

- **CREATE TABLE**: Tworzy nową, pustą tabelę: nazwy kolumn, ich typy i reguły. Działa tak samo w obu systemach.
- **Typy danych**: INT: liczba całkowita; SMALLINT: mała liczba całkowita; BIGINT: duża liczba całkowita; CHAR(n): tekst o stałej długości; VARCHAR(n): tekst do n znaków; DATE: sama data; NUMERIC(p,s): liczba dokładna z s miejscami po przecinku (dobra do pieniędzy). Te typy są takie same w obu systemach.
- **PRIMARY KEY (klucz główny)**: Kolumna (lub zestaw kolumn), która jednoznacznie identyfikuje wiersz. Nie może się powtarzać ani być pusta. Baza sama zakłada na niej indeks, więc wyszukiwanie po kluczu jest szybkie.
- **REFERENCES (klucz obcy, FOREIGN KEY)**: Kolumna wskazuje na klucz główny innej tabeli. Baza nie pozwoli wpisać wartości, której nie ma w tabeli nadrzędnej (np. transakcji klienta, który nie istnieje). Składnia identyczna w obu systemach.
- **NOT NULL**: Kolumna nie może być pusta (NULL = brak wartości).
- **DEFAULT**: Wartość wpisywana automatycznie, gdy przy dodawaniu wiersza nie podamy tej kolumny.
- **BOOLEAN vs BIT**: PostgreSQL ma prawdziwy typ logiczny BOOLEAN z wartościami true / false. SQL Server go nie ma: używa BIT z wartościami 1 / 0. Dlatego w T-SQL zawsze piszemy porównanie, np. is_fraud = 1, a w PostgreSQL wystarczy samo is_fraud.
- **TIMESTAMP vs DATETIME2**: W PostgreSQL TIMESTAMP to data z godziną. Uwaga, pułapka: w SQL Server TIMESTAMP oznacza coś zupełnie innego (licznik wersji wiersza, synonim ROWVERSION). Odpowiednikiem jest DATETIME2.

3. Kontrola: lista utworzonych tabel

Co robi: Pyta systemowy katalog bazy o listę tabel w domyślnym schemacie. Służy do sprawdzenia, czy wszystkie 6 tabel powstało (i czy w dobrej bazie).

Wynik na Twoich danych: 6 nazw tabel: dim_customer, dim_device, dim_fx_rate, dim_merchant, fact_customer_risk, fact_transaction.

PostgreSQL (Twój skrypt)

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;
```

Microsoft SQL Server (SSMS)

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'dbo'
ORDER BY table_name;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`'public'`	`'dbo'`	Domyślny schemat tabel: w PostgreSQL public, w SQL Server dbo.

Funkcje i metody użyte w tym zapytaniu

- **SELECT ... FROM ... WHERE**: Podstawa: SELECT wybiera kolumny, FROM wskazuje tabelę, WHERE filtruje wiersze przed liczeniem.
- **information_schema.tables**: Systemowy widok ze spisem tabel w bazie. Istnieje w obu systemach. Domyślny schemat (folder na tabele) w PostgreSQL to public, a w SQL Server dbo.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

01_data_quality.sql

1. 17 kontroli jakości danych w jednej tabeli

Co robi: Każda kontrola to jedno małe zapytanie, które liczy „złe” wiersze (0 = OK). UNION ALL skleja je w jedną tabelę wyników, a CASE na końcu nadaje status OK / REVIEW / ERROR. Sprawdzamy duplikaty, puste pola, zakresy wartości, spójność dat, osierocone rekordy (klucz bez pary) i wartości odstające. Krok tx_eur przelicza kwotę na EUR po kursie z dnia transakcji.

Wynik na Twoich danych: 16 kontroli OK, jedna REVIEW: 258 transakcji powyżej 99,9 percentyla kwoty (kandydaci do analizy, nie błędy).

PostgreSQL (Twój skrypt)

```
WITH tx_eur AS (
    -- transaction amount converted to EUR at the rate from the transaction day
    SELECT t.*, t.amount * fx.eur_rate AS amount_eur
    FROM fact_transaction t
    LEFT JOIN dim_fx_rate fx
           ON fx.rate_date = t.tx_datetime::date      -- T-SQL: CAST(t.tx_datetime AS date)
          AND fx.currency  = t.currency
),
checks AS (
    -- 1. Duplicate keys
    SELECT 1 AS check_no, 'Duplicate check' AS category, 'Duplicated transaction_id' AS check_name,
           (SELECT COUNT(*) - COUNT(DISTINCT transaction_id) FROM fact_transaction) AS failed_rows
    UNION ALL
    SELECT 2, 'Duplicate check', 'Duplicated customer_id',
           (SELECT COUNT(*) - COUNT(DISTINCT customer_id) FROM dim_customer)

    -- 2. Completeness of key fields
    UNION ALL
    SELECT 3, 'Null check', 'Transactions with empty customer_id / amount / tx_datetime / currency',
           (SELECT COUNT(*) FROM fact_transaction
            WHERE customer_id IS NULL OR amount IS NULL OR tx_datetime IS NULL OR currency IS NULL)
    UNION ALL
    SELECT 4, 'Null check', 'Transactions without merchant_id or device_id',
           (SELECT COUNT(*) FROM fact_transaction WHERE merchant_id IS NULL OR device_id IS NULL)
    UNION ALL
    SELECT 5, 'Null check', 'Customers without country, age or signup date',
           (SELECT COUNT(*) FROM dim_customer WHERE country IS NULL OR age IS NULL OR signup_date IS NULL)

    -- 3. Value ranges
    UNION ALL
    SELECT 6, 'Range check', 'Amount <= 0',
           (SELECT COUNT(*) FROM fact_transaction WHERE amount <= 0)
    UNION ALL
    SELECT 7, 'Range check', 'Age outside 18-100',
           (SELECT COUNT(*) FROM dim_customer WHERE age NOT BETWEEN 18 AND 100)
    UNION ALL
    SELECT 8, 'Range check', 'Transactions outside the period 2026-01-01 - 2026-06-30',
           (SELECT COUNT(*) FROM fact_transaction
            WHERE tx_datetime < DATE '2026-01-01' OR tx_datetime >= DATE '2026-07-01')
    UNION ALL
    SELECT 9, 'Range check', 'Transactions without an exchange rate for that day',
           (SELECT COUNT(*) FROM tx_eur WHERE amount_eur IS NULL)

    -- 4. Date consistency
    UNION ALL
    SELECT 10, 'Date consistency', 'Transaction before the customer signup date',
           (SELECT COUNT(*) FROM fact_transaction t
            JOIN dim_customer c USING (customer_id)
            WHERE t.tx_datetime::date < c.signup_date)
    UNION ALL
    SELECT 11, 'Date consistency', 'Transaction outside the device activity window (first_seen - last_seen)',
           (SELECT COUNT(*) FROM fact_transaction t
            JOIN dim_device d USING (device_id)
            WHERE t.tx_datetime NOT BETWEEN d.first_seen AND d.last_seen)
    UNION ALL
    SELECT 12, 'Date consistency', 'Device with first_seen later than last_seen',
           (SELECT COUNT(*) FROM dim_device WHERE first_seen > last_seen)

    -- 5. Referential integrity (orphan records)
    UNION ALL
    SELECT 13, 'Referential integrity', 'Transactions with a non-existent customer',
           (SELECT COUNT(*) FROM fact_transaction t
            WHERE NOT EXISTS (SELECT 1 FROM dim_customer c WHERE c.customer_id = t.customer_id))
    UNION ALL
    SELECT 14, 'Referential integrity', 'Transactions with a non-existent merchant',
           (SELECT COUNT(*) FROM fact_transaction t
            WHERE NOT EXISTS (SELECT 1 FROM dim_merchant m WHERE m.merchant_id = t.merchant_id))
```

```
    UNION ALL
    SELECT 15, 'Referential integrity', 'Transactions with a non-existent device',
           (SELECT COUNT(*) FROM fact_transaction t
            WHERE NOT EXISTS (SELECT 1 FROM dim_device d WHERE d.device_id = t.device_id))
    UNION ALL
    SELECT 16, 'Referential integrity', 'Transactions from a device belonging to another customer',
           (SELECT COUNT(*) FROM fact_transaction t
            JOIN dim_device d USING (device_id)
            WHERE d.customer_id <> t.customer_id)

    -- 6. Outliers (not an error, just candidates for further analysis)
    UNION ALL
    SELECT 17, 'Outlier check', 'Transactions above the 99.9th percentile of EUR amount (to review, not to delete)',
           (SELECT COUNT(*) FROM tx_eur
            WHERE amount_eur > (SELECT PERCENTILE_CONT(0.999) WITHIN GROUP (ORDER BY amount_eur) FROM tx_eur))
)
SELECT check_no, category, check_name, failed_rows,
       CASE WHEN failed_rows = 0 THEN 'OK'
            WHEN category = 'Outlier check' THEN 'REVIEW'
            ELSE 'ERROR' END AS status
FROM checks
ORDER BY check_no;
```

Microsoft SQL Server (SSMS)

```
WITH tx_eur AS (
    SELECT t.*, t.amount * fx.eur_rate AS amount_eur
    FROM fact_transaction t
    LEFT JOIN dim_fx_rate fx
           ON fx.rate_date = CAST(t.tx_datetime AS date)
          AND fx.currency  = t.currency
),
checks AS (
    -- 1. Duplicate keys
    SELECT 1 AS check_no, 'Duplicate check' AS category, 'Duplicated transaction_id' AS check_name,
           (SELECT COUNT(*) - COUNT(DISTINCT transaction_id) FROM fact_transaction) AS failed_rows
    UNION ALL
    SELECT 2, 'Duplicate check', 'Duplicated customer_id',
           (SELECT COUNT(*) - COUNT(DISTINCT customer_id) FROM dim_customer)

    -- 2. Completeness of key fields
    UNION ALL
    SELECT 3, 'Null check', 'Transactions with empty customer_id / amount / tx_datetime / currency',
           (SELECT COUNT(*) FROM fact_transaction
            WHERE customer_id IS NULL OR amount IS NULL OR tx_datetime IS NULL OR currency IS NULL)
    UNION ALL
    SELECT 4, 'Null check', 'Transactions without merchant_id or device_id',
           (SELECT COUNT(*) FROM fact_transaction WHERE merchant_id IS NULL OR device_id IS NULL)
    UNION ALL
    SELECT 5, 'Null check', 'Customers without country, age or signup date',
           (SELECT COUNT(*) FROM dim_customer WHERE country IS NULL OR age IS NULL OR signup_date IS NULL)

    -- 3. Value ranges
    UNION ALL
    SELECT 6, 'Range check', 'Amount <= 0',
           (SELECT COUNT(*) FROM fact_transaction WHERE amount <= 0)
    UNION ALL
    SELECT 7, 'Range check', 'Age outside 18-100',
           (SELECT COUNT(*) FROM dim_customer WHERE age NOT BETWEEN 18 AND 100)
    UNION ALL
    SELECT 8, 'Range check', 'Transactions outside the period 2026-01-01 - 2026-06-30',
           (SELECT COUNT(*) FROM fact_transaction
            WHERE tx_datetime < '2026-01-01' OR tx_datetime >= '2026-07-01')
    UNION ALL
    SELECT 9, 'Range check', 'Transactions without an exchange rate for that day',
           (SELECT COUNT(*) FROM tx_eur WHERE amount_eur IS NULL)

    -- 4. Date consistency
    UNION ALL
    SELECT 10, 'Date consistency', 'Transaction before the customer signup date',
           (SELECT COUNT(*) FROM fact_transaction t
            JOIN dim_customer c ON c.customer_id = t.customer_id
            WHERE CAST(t.tx_datetime AS date) < c.signup_date)
    UNION ALL
    SELECT 11, 'Date consistency', 'Transaction outside the device activity window (first_seen - last_seen)',
           (SELECT COUNT(*) FROM fact_transaction t
            JOIN dim_device d ON d.device_id = t.device_id
            WHERE t.tx_datetime NOT BETWEEN d.first_seen AND d.last_seen)
    UNION ALL
    SELECT 12, 'Date consistency', 'Device with first_seen later than last_seen',
           (SELECT COUNT(*) FROM dim_device WHERE first_seen > last_seen)

    -- 5. Referential integrity (orphan records)
    UNION ALL
```

```
    SELECT 13, 'Referential integrity', 'Transactions with a non-existent customer',
           (SELECT COUNT(*) FROM fact_transaction t
            WHERE NOT EXISTS (SELECT 1 FROM dim_customer c WHERE c.customer_id = t.customer_id))
    UNION ALL
    SELECT 14, 'Referential integrity', 'Transactions with a non-existent merchant',
           (SELECT COUNT(*) FROM fact_transaction t
            WHERE NOT EXISTS (SELECT 1 FROM dim_merchant m WHERE m.merchant_id = t.merchant_id))
    UNION ALL
    SELECT 15, 'Referential integrity', 'Transactions with a non-existent device',
           (SELECT COUNT(*) FROM fact_transaction t
            WHERE NOT EXISTS (SELECT 1 FROM dim_device d WHERE d.device_id = t.device_id))
    UNION ALL
    SELECT 16, 'Referential integrity', 'Transactions from a device belonging to another customer',
           (SELECT COUNT(*) FROM fact_transaction t
            JOIN dim_device d ON d.device_id = t.device_id
            WHERE d.customer_id <> t.customer_id)

    -- 6. Outliers (not an error, just candidates for further analysis)
    UNION ALL
    SELECT 17, 'Outlier check', 'Transactions above the 99.9th percentile of EUR amount (to review, not to delete)',
           (SELECT COUNT(*) FROM tx_eur
            WHERE amount_eur > (SELECT TOP 1 PERCENTILE_CONT(0.999) WITHIN GROUP (ORDER BY amount_eur) OVER ()
                                FROM tx_eur))
)
SELECT check_no, category, check_name, failed_rows,
       CASE WHEN failed_rows = 0 THEN 'OK'
            WHEN category = 'Outlier check' THEN 'REVIEW'
            ELSE 'ERROR' END AS status
FROM checks
ORDER BY check_no;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`t.tx_datetime::date`	`CAST(t.tx_datetime AS date)`	Obcięcie godziny, żeby dopasować kurs z danego dnia.
`DATE '2026-01-01'`	`'2026-01-01'`	SQL Server sam zamienia tekst na datę przy porównaniu z kolumną daty.
`JOIN ... USING (customer_id)`	`JOIN ... ON c.customer_id = t.customer_id`	T-SQL nie zna USING, warunek łączenia trzeba wypisać.
`PERCENTILE_CONT(...) WITHIN GROUP (...)`	`TOP 1 PERCENTILE_CONT(...) WITHIN GROUP (...) OVER ()`	W SQL Server percentyl jest tylko funkcją okna: OVER () liczy go na całej tabeli i dopisuje do każdego wiersza, a TOP 1 bierze jedną wartość.

Funkcje i metody użyte w tym zapytaniu

- **WITH nazwa AS (...) (CTE)**: „Tymczasowa tabela na czas jednego zapytania". Dzielimy duże zapytanie na nazwane kroki, a każdy kolejny krok może korzystać z poprzednich. Działa tak samo w obu systemach.
- **LEFT JOIN**: Jak JOIN, ale zostawia wszystkie wiersze z lewej tabeli; tam, gdzie nie ma pary, kolumny z prawej tabeli są puste (NULL). Używamy np. do znalezienia transakcji bez kursu waluty.
- **::date vs CAST(... AS date)**: Zamiana typu. W PostgreSQL skrót x::date (tu: obcina godzinę z daty-czasu). W SQL Server CAST(x AS date) lub CONVERT(date, x). Zapis CAST działa też w PostgreSQL.
- **AS (alias)**: Nadaje kolumnie lub tabeli krótką nazwę. t.customer_id znaczy: kolumna customer_id z tabeli o aliasie t.
- **SELECT * / alias.***: Wszystkie kolumny tabeli lub kroku CTE.
- **UNION ALL**: Skleja wyniki kilku zapytań jeden pod drugim (bez usuwania duplikatów). Każde zapytanie musi mieć tyle samo kolumn.
- **Podzapytanie skalarne (SELECT ...)**: Zapytanie w nawiasie, które zwraca jedną liczbę i można go użyć jak zwykłej wartości w kolumnie lub warunku.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(DISTINCT x)**: Liczy różne wartości, np. ile różnych urządzeń albo dni. COUNT(*) − COUNT(DISTINCT id) pokazuje liczbę duplikatów.
- **IS NULL / IS NOT NULL**: Sprawdza, czy pole jest puste. Nie można pisać = NULL, bo porównanie z NULL nigdy nie daje „prawdy".
- **AND / OR / NOT**: Łączenie warunków: oba naraz (AND), choć jeden (OR), zaprzeczenie (NOT). W PostgreSQL NOT is_fraud działa na kolumnie BOOLEAN; w T-SQL piszemy is_fraud = 0.
- **BETWEEN a AND b / NOT BETWEEN**: Czy wartość leży w przedziale, włącznie z końcami. NOT BETWEEN odwraca warunek.
- **Stała data: DATE '2026-01-01'**: W PostgreSQL DATE '...' i TIMESTAMP '...' tworzą wartość daty z tekstu. W SQL Server wystarczy tekst '2026-01-01' (porównywany z kolumną daty) albo CAST('2026-01-01' AS date).

- **JOIN ... ON**: Łączy wiersze z dwóch tabel według warunku, np. transakcja z kursem waluty z tego samego dnia. Zostają tylko wiersze, które mają parę (INNER JOIN).
- **JOIN ... USING (kolumna)**: Skrót w PostgreSQL: gdy obie tabele mają kolumnę o tej samej nazwie, USING (customer_id) znaczy ON a.customer_id = b.customer_id. SQL Server nie zna USING, więc trzeba pisać pełne ON.
- **NOT EXISTS (podzapytanie)**: Prawda, gdy podzapytanie nie zwraca żadnego wiersza. Tu: transakcja, dla której nie ma klienta / sprzedawcy / urządzenia, czyli „osierocony" rekord. Tak samo w obu systemach.
- **PERCENTILE_CONT(p) WITHIN GROUP (ORDER BY x)**: Percentyl z interpolacją: 0.5 to mediana, 0.999 to wartość, poniżej której jest 99,9% danych. W PostgreSQL działa jak zwykła funkcja agregująca (z GROUP BY). W SQL Server tylko jako funkcja okna: trzeba dopisać OVER (PARTITION BY ...) lub OVER () i usunąć powtórzenia (DISTINCT / TOP 1).
- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli": dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

2. Outliery: transakcje 10× wyższe niż mediana klienta

Co robi: Dla każdego klienta liczy medianę jego kwot w EUR (typową transakcję), a potem szuka transakcji co najmniej 10 razy wyższych. Bierze tylko klientów z historią (min. 5 transakcji). To pierwszy przykład, że „nietypowe" nie zawsze znaczy „oszustwo".

Wynik na Twoich danych: 50 transakcji: 31 to fraud, 19 to legalne duże zakupy.

PostgreSQL (Twój skrypt)

```
WITH tx_eur AS (
    SELECT t.transaction_id, t.customer_id, t.tx_datetime, t.amount, t.currency, t.is_fraud,
           ROUND(t.amount * fx.eur_rate, 2) AS amount_eur
    FROM fact_transaction t
    JOIN dim_fx_rate fx ON fx.rate_date = t.tx_datetime::date AND fx.currency = t.currency
),
customer_median AS (
    SELECT customer_id,
           PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount_eur) AS median_eur,   -- T-SQL: PERCENTILE_CONT(...)
OVER (PARTITION BY ...)
           COUNT(*) AS tx_count
    FROM tx_eur
    GROUP BY customer_id
)
SELECT e.transaction_id, e.customer_id, e.tx_datetime, e.amount, e.currency, e.amount_eur,
       ROUND(m.median_eur::numeric, 2)                AS customer_median_eur,
       ROUND((e.amount_eur / m.median_eur)::numeric, 1) AS times_higher,
       e.is_fraud
FROM tx_eur e
JOIN customer_median m USING (customer_id)
WHERE m.tx_count >= 5                          -- only customers with history
  AND e.amount_eur > 10 * m.median_eur         -- at least 10x more than usual
ORDER BY times_higher DESC
LIMIT 50;                                      -- T-SQL: SELECT TOP 50
```

Microsoft SQL Server (SSMS)

```
WITH tx_eur AS (
    SELECT t.transaction_id, t.customer_id, t.tx_datetime, t.amount, t.currency, t.is_fraud,
           ROUND(t.amount * fx.eur_rate, 2) AS amount_eur
    FROM fact_transaction t
    JOIN dim_fx_rate fx ON fx.rate_date = CAST(t.tx_datetime AS date) AND fx.currency = t.currency
),
customer_median AS (
    SELECT DISTINCT customer_id,
           PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount_eur)
               OVER (PARTITION BY customer_id) AS median_eur,
           COUNT(*) OVER (PARTITION BY customer_id) AS tx_count
    FROM tx_eur
)
SELECT TOP 50 e.transaction_id, e.customer_id, e.tx_datetime, e.amount, e.currency, e.amount_eur,
       ROUND(m.median_eur, 2)                AS customer_median_eur,
       ROUND(e.amount_eur / m.median_eur, 1)  AS times_higher,
       e.is_fraud
FROM tx_eur e
JOIN customer_median m ON m.customer_id = e.customer_id
WHERE m.tx_count >= 5                          -- only customers with history
  AND e.amount_eur > 10 * m.median_eur         -- at least 10x more than usual
ORDER BY times_higher DESC;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`PERCENTILE_CONT ... + GROUP BY`	`PERCENTILE_CONT ... OVER (PARTITION BY customer_id) + DISTINCT`	Mediana jako funkcja okna dopisuje się do każdej transakcji klienta; DISTINCT zostawia jeden wiersz na klienta. COUNT(*) też liczymy oknem, bo nie ma GROUP BY.
`x::numeric w ROUND`	`samo x`	PostgreSQL wymaga typu numeric w ROUND(x, n), a percentyl zwraca double. W SQL Server ROUND działa na każdym typie.
`LIMIT 50 na końcu`	`SELECT TOP 50 na początku`	Ograniczenie liczby wierszy.
`USING (customer_id)`	`ON m.customer_id = e.customer_id`	Brak USING w T-SQL.

Funkcje i metody użyte w tym zapytaniu

- **WITH nazwa AS (...) (CTE)**: „Tymczasowa tabela na czas jednego zapytania”. Dzielimy duże zapytanie na nazwane kroki, a każdy kolejny krok może korzystać z poprzednich. Działa tak samo w obu systemach.
- **JOIN ... ON**: Łączy wiersze z dwóch tabel według warunku, np. transakcja z kursem waluty z tego samego dnia. Zostają tylko wiersze, które mają parę (INNER JOIN).
- **::date vs CAST(... AS date)**: Zamiana typu. W PostgreSQL skrót x::date (tu: obcina godzinę z daty-czasu). W SQL Server CAST(x AS date) lub CONVERT(date, x). Zapis CAST działa też w PostgreSQL.
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **PERCENTILE_CONT(p) WITHIN GROUP (ORDER BY x)**: Percentyl z interpolacją: 0.5 to mediana, 0.999 to wartość, poniżej której jest 99,9% danych. W PostgreSQL działa jak zwykła funkcja agregująca (z GROUP BY). W SQL Server tylko jako funkcja okna: trzeba dopisać OVER (PARTITION BY ...) lub OVER () i usunąć powtórzenia (DISTINCT / TOP 1).
- **Funkcja okna: ... OVER (PARTITION BY ... ORDER BY ...)**: Liczy wartość dla każdego wiersza na podstawie „okna” innych wierszy, ale nie skleja wierszy w grupy (w przeciwieństwie do GROUP BY). PARTITION BY dzieli dane na osobne okna (np. osobno dla każdego klienta), ORDER BY ustala kolejność w oknie. Składnia jest taka sama w obu systemach.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **JOIN ... USING (kolumna)**: Skrót w PostgreSQL: gdy obie tabele mają kolumnę o tej samej nazwie, USING (customer_id) znaczy ON a.customer_id = b.customer_id. SQL Server nie zna USING, więc trzeba pisać pełne ON.
- **::typ vs CAST(... AS typ)**: Zamiana typu, np. ::numeric, ::int, ::text. W SQL Server CAST(x AS typ). Tekstowy odpowiednik ::text to CAST(x AS VARCHAR(10)).
- **100.0 * a / b (dzielenie z ułamkiem)**: Dzielenie dwóch liczb całkowitych daje w obu systemach wynik całkowity (7/2 = 3). Mnożenie przez 100.0 lub 1.0 zamienia liczbę na dziesiętną i wynik ma ułamek.
- **LIMIT n vs TOP n**: Zwraca tylko pierwsze n wierszy (po sortowaniu). PostgreSQL: LIMIT n na końcu zapytania. SQL Server: SELECT TOP n tuż po SELECT.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

02_sql_analysis/01_fraud_rate.sql

1. Ogólny fraud rate (liczbowo i kwotowo)

Co robi: Jedna linijka podsumowania całego okresu: ile jest transakcji, ile fraudów, jaki to procent, ile wynosi obrót i strata w EUR oraz jaki procent wartości to fraud. Kwoty przeliczamy na EUR przez kurs z dnia transakcji.

Wynik na Twoich danych: 257 947 transakcji, 4 242 fraudy = 1,64% liczby, ale 14,65% wartości (3,43 mln z 23,4 mln EUR).

PostgreSQL (Twój skrypt)

```
SELECT
    COUNT(*)                                                AS transactions,
    COUNT(*) FILTER (WHERE t.is_fraud)                      AS fraud_transactions,
    ROUND(100.0 * AVG(t.is_fraud::int), 2)                  AS fraud_rate_pct,
    ROUND(SUM(t.amount * fx.eur_rate), 0)                   AS total_eur,
    ROUND(SUM(t.amount * fx.eur_rate) FILTER (WHERE t.is_fraud), 0) AS fraud_loss_eur,
    ROUND(100.0 * SUM(t.amount * fx.eur_rate) FILTER (WHERE t.is_fraud)
                / SUM(t.amount * fx.eur_rate), 2)           AS fraud_rate_by_amount_pct
FROM fact_transaction t
JOIN dim_fx_rate fx ON fx.rate_date = t.tx_datetime::date AND fx.currency = t.currency;
```

Microsoft SQL Server (SSMS)

```
SELECT
    COUNT(*)                                                AS transactions,
    SUM(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END)         AS fraud_transactions,
    ROUND(100.0 * AVG(CAST(t.is_fraud AS float)), 2)        AS fraud_rate_pct,
    ROUND(SUM(t.amount * fx.eur_rate), 0)                   AS total_eur,
    ROUND(SUM(CASE WHEN t.is_fraud = 1 THEN t.amount * fx.eur_rate END), 0) AS fraud_loss_eur,
    ROUND(100.0 * SUM(CASE WHEN t.is_fraud = 1 THEN t.amount * fx.eur_rate END)
                / SUM(t.amount * fx.eur_rate), 2)           AS fraud_rate_by_amount_pct
FROM fact_transaction t
JOIN dim_fx_rate fx ON fx.rate_date = CAST(t.tx_datetime AS date) AND fx.currency = t.currency;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`COUNT(*) FILTER (WHERE t.is_fraud)`	`SUM(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END)`	Liczenie tylko fraudów. W T-SQL warunek na BIT musi mieć = 1.
`AVG(t.is_fraud::int)`	`AVG(CAST(t.is_fraud AS float))`	Średnia z 0/1 = udział fraudów. Rzut na float, bo AVG z int w T-SQL obcina ułamek.
`SUM(x) FILTER (WHERE ...)`	`SUM(CASE WHEN ... THEN x END)`	Suma kwot tylko z fraudów; pozostałe wiersze dają NULL, który SUM pomija.
`::date`	`CAST(... AS date)`	Zamiana typu.

Funkcje i metody użyte w tym zapytaniu

- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AVG(is_fraud::int) vs AVG(CAST(is_fraud AS float))**: Średnia z 0/1 to udział jedynek, czyli fraud rate. W PostgreSQL zamieniamy BOOLEAN na liczbę (::int). W SQL Server BIT trzeba zamienić na float, bo AVG z liczby całkowitej dałoby wynik całkowity (0).
- **100.0 * a / b (dzielenie z ułamkiem)**: Dzielenie dwóch liczb całkowitych daje w obu systemach wynik całkowity (7/2 = 3). Mnożenie przez 100.0 lub 1.0 zamienia liczbę na dziesiętną i wynik ma ułamek.
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **SUM(x)**: Suma wartości w grupie.
- **JOIN ... ON**: Łączy wiersze z dwóch tabel według warunku, np. transakcja z kursem waluty z tego samego dnia. Zostają tylko wiersze, które mają parę (INNER JOIN).
- **::date vs CAST(... AS date)**: Zamiana typu. W PostgreSQL skrót x::date (tu: obcina godzinę z daty-czasu). W SQL Server CAST(x AS date) lub CONVERT(date, x). Zapis CAST działa też w PostgreSQL.
- **AS (alias)**: Nadaje kolumnie lub tabeli krótką nazwę. t.customer_id znaczy: kolumna customer_id z tabeli o aliasie t.

2. Fraud rate według kategorii sprzedawcy

Co robi: Grupuje transakcje po kategorii sprzedawcy (crypto, electronics, grocery...) i dla każdej liczy liczbę transakcji, fraudów, fraud rate i stratę w EUR. Sortuje od najbardziej ryzykownej.

Wynik na Twoich danych: crypto 18,97%, electronics 10,21%, gift_cards 8,40%; online_shopping 1,23%; 5 kategorii ma 0% (efekt danych syntetycznych).

PostgreSQL (Twój skrypt)

```
SELECT
    m.merchant_category,
    COUNT(*)                                          AS transactions,
    COUNT(*) FILTER (WHERE t.is_fraud)                AS fraud_transactions,
    ROUND(100.0 * AVG(t.is_fraud::int), 2)            AS fraud_rate_pct,
    COALESCE(ROUND(SUM(t.amount * fx.eur_rate) FILTER (WHERE t.is_fraud), 0), 0) AS fraud_loss_eur   -- T-SQL:
ISNULL(...)
FROM fact_transaction t
JOIN dim_merchant m USING (merchant_id)
JOIN dim_fx_rate fx ON fx.rate_date = t.tx_datetime::date AND fx.currency = t.currency
GROUP BY m.merchant_category
ORDER BY fraud_rate_pct DESC;
```

Microsoft SQL Server (SSMS)

```
SELECT
    m.merchant_category,
    COUNT(*)                                          AS transactions,
    SUM(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END)   AS fraud_transactions,
    ROUND(100.0 * AVG(CAST(t.is_fraud AS float)), 2)  AS fraud_rate_pct,
    ISNULL(ROUND(SUM(CASE WHEN t.is_fraud = 1 THEN t.amount * fx.eur_rate END), 0), 0) AS fraud_loss_eur
FROM fact_transaction t
JOIN dim_merchant m ON m.merchant_id = t.merchant_id
JOIN dim_fx_rate fx ON fx.rate_date = CAST(t.tx_datetime AS date) AND fx.currency = t.currency
GROUP BY m.merchant_category
ORDER BY fraud_rate_pct DESC;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`FILTER / AVG(::int)`	`SUM(CASE ...) / AVG(CAST AS float)`	Jak w zapytaniu 1.
`COALESCE`	`ISNULL`	Kategorie bez fraudu mają NULL zamiast straty, zamieniamy go na 0. COALESCE zadziała też w T-SQL; ISNULL to wersja typowa dla SQL Server.
`USING (merchant_id)`	`ON m.merchant_id = t.merchant_id`	Brak USING w T-SQL.

Funkcje i metody użyte w tym zapytaniu

- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AVG(is_fraud::int) vs AVG(CAST(is_fraud AS float))**: Średnia z 0/1 to udział jedynek, czyli fraud rate. W PostgreSQL zamieniamy BOOLEAN na liczbę (::int). W SQL Server BIT trzeba zamienić na float, bo AVG z liczby całkowitej dałoby wynik całkowity (0).
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **COALESCE(x, y) / ISNULL(x, y)**: Zwraca pierwszą niepustą wartość, np. 0 zamiast NULL. COALESCE działa w obu systemach; ISNULL to wersja tylko z SQL Server (dokładnie 2 argumenty).
- **JOIN ... ON**: Łączy wiersze z dwóch tabel według warunku, np. transakcja z kursem waluty z tego samego dnia. Zostają tylko wiersze, które mają parę (INNER JOIN).
- **JOIN ... USING (kolumna)**: Skrót w PostgreSQL: gdy obie tabele mają kolumnę o tej samej nazwie, USING (customer_id) znaczy ON a.customer_id = b.customer_id. SQL Server nie zna USING, więc trzeba pisać pełne ON.
- **::date vs CAST(... AS date)**: Zamiana typu. W PostgreSQL skrót x::date (tu: obcina godzinę z daty-czasu). W SQL Server CAST(x AS date) lub CONVERT(date, x). Zapis CAST działa też w PostgreSQL.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

3. Fraud rate według kraju transakcji

Co robi: Grupuje transakcje po kraju, w którym się odbyły, i pokazuje fraud rate. HAVING odrzuca kraje z mniej niż 500 transakcjami, bo przy małej liczbie procenty są przypadkowe.

Wynik na Twoich danych: Kraje spoza Europy mają dużo wyższy fraud rate (CN 42,59%), Europa poniżej 1,2%.

PostgreSQL (Twój skrypt)

```
SELECT
    t.country,
    COUNT(*)                                        AS transactions,
    COUNT(*) FILTER (WHERE t.is_fraud)              AS fraud_transactions,
    ROUND(100.0 * AVG(t.is_fraud::int), 2)          AS fraud_rate_pct
FROM fact_transaction t
GROUP BY t.country
HAVING COUNT(*) >= 500
ORDER BY fraud_rate_pct DESC;
```

Microsoft SQL Server (SSMS)

```
SELECT
    t.country,
    COUNT(*)                                        AS transactions,
    SUM(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END) AS fraud_transactions,
    ROUND(100.0 * AVG(CAST(t.is_fraud AS float)), 2) AS fraud_rate_pct
FROM fact_transaction t
GROUP BY t.country
HAVING COUNT(*) >= 500
ORDER BY fraud_rate_pct DESC;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`FILTER / AVG(::int)`	`SUM(CASE ...) / AVG(CAST AS float)`	Jedyna różnica: liczenie fraudów i ich udziału.

Funkcje i metody użyte w tym zapytaniu

- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **HAVING**: Filtr na grupach, działa po GROUP BY (np. tylko kraje z co najmniej 500 transakcjami). WHERE filtruje wiersze przed grupowaniem, HAVING grupy po.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AVG(is_fraud::int) vs AVG(CAST(is_fraud AS float))**: Średnia z 0/1 to udział jedynek, czyli fraud rate. W PostgreSQL zamieniamy BOOLEAN na liczbę (::int). W SQL Server BIT trzeba zamienić na float, bo AVG z liczby całkowitej dałoby wynik całkowity (0).
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

4. Transakcja za granicą a ryzyko

Co robi: Porównuje kraj transakcji z krajem klienta. CASE dzieli transakcje na dwie grupy: „home country” i „abroad”, a potem liczymy fraud rate w każdej.

Wynik na Twoich danych: Za granicą 10,14% fraudu, w kraju klienta 0,11%.

PostgreSQL (Twój skrypt)

```
SELECT
    CASE WHEN t.country = c.country THEN 'home country' ELSE 'abroad' END AS location,
    COUNT(*)                                             AS transactions,
    COUNT(*) FILTER (WHERE t.is_fraud)                   AS fraud_transactions,
    ROUND(100.0 * AVG(t.is_fraud::int), 2)               AS fraud_rate_pct
FROM fact_transaction t
JOIN dim_customer c USING (customer_id)
GROUP BY 1                                          -- T-SQL: GROUP BY needs the full CASE expression
ORDER BY fraud_rate_pct DESC;
```

Microsoft SQL Server (SSMS)

```
SELECT
    CASE WHEN t.country = c.country THEN 'home country' ELSE 'abroad' END AS location,
    COUNT(*)                                             AS transactions,
    SUM(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END)      AS fraud_transactions,
    ROUND(100.0 * AVG(CAST(t.is_fraud AS float)), 2)     AS fraud_rate_pct
FROM fact_transaction t
JOIN dim_customer c ON c.customer_id = t.customer_id
GROUP BY CASE WHEN t.country = c.country THEN 'home country' ELSE 'abroad' END
ORDER BY fraud_rate_pct DESC;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`GROUP BY 1`	`GROUP BY CASE ... END (całe wyrażenie)`	SQL Server nie pozwala grupować po numerze kolumny ani po aliasie z SELECT-a.
`USING (customer_id)`	`ON c.customer_id = t.customer_id`	Brak USING w T-SQL.
`FILTER / AVG(::int)`	`SUM(CASE ...) / AVG(CAST AS float)`	Liczenie fraudów.

Funkcje i metody użyte w tym zapytaniu

- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli”: dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **JOIN ... ON**: Łączy wiersze z dwóch tabel według warunku, np. transakcja z kursem waluty z tego samego dnia. Zostają tylko wiersze, które mają parę (INNER JOIN).
- **JOIN ... USING (kolumna)**: Skrót w PostgreSQL: gdy obie tabele mają kolumnę o tej samej nazwie, USING (customer_id) znaczy ON a.customer_id = b.customer_id. SQL Server nie zna USING, więc trzeba pisać pełne ON.
- **GROUP BY 1 (numer kolumny)**: W PostgreSQL można grupować po numerze kolumny z SELECT-a. SQL Server na to nie pozwala: trzeba powtórzyć całe wyrażenie (np. cały CASE) albo policzyć je wcześniej w podzapytaniu / CROSS APPLY i grupować po aliasie.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AVG(is_fraud::int) vs AVG(CAST(is_fraud AS float))**: Średnia z 0/1 to udział jedynek, czyli fraud rate. W PostgreSQL zamieniamy BOOLEAN na liczbę (::int). W SQL Server BIT trzeba zamienić na float, bo AVG z liczby całkowitej dałoby wynik całkowity (0).
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

02_sql_analysis/02_fraud_trend.sql

1. Trend miesięczny

Co robi: Obcina datę każdej transakcji do pierwszego dnia miesiąca i grupuje po niej. Dla każdego miesiąca: liczba transakcji, fraudów, fraud rate i strata w EUR.

Wynik na Twoich danych: Styczeń: 42 334 transakcji, 563 fraudy (1,33%); kolejne miesiące 1,65–1,81%.

PostgreSQL (Twój skrypt)

```
SELECT
    DATE_TRUNC('month', t.tx_datetime)::date            AS month,
    COUNT(*)                                            AS transactions,
    COUNT(*) FILTER (WHERE t.is_fraud)                  AS fraud_transactions,
    ROUND(100.0 * AVG(t.is_fraud::int), 2)              AS fraud_rate_pct,
    ROUND(SUM(t.amount * fx.eur_rate) FILTER (WHERE t.is_fraud), 0) AS fraud_loss_eur
FROM fact_transaction t
JOIN dim_fx_rate fx ON fx.rate_date = t.tx_datetime::date AND fx.currency = t.currency
GROUP BY 1
ORDER BY 1;
```

Microsoft SQL Server (SSMS)

```
SELECT
    DATEFROMPARTS(YEAR(t.tx_datetime), MONTH(t.tx_datetime), 1) AS month,
    COUNT(*)                                            AS transactions,
    SUM(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END)     AS fraud_transactions,
    ROUND(100.0 * AVG(CAST(t.is_fraud AS float)), 2)    AS fraud_rate_pct,
    ROUND(SUM(CASE WHEN t.is_fraud = 1 THEN t.amount * fx.eur_rate END), 0) AS fraud_loss_eur
FROM fact_transaction t
JOIN dim_fx_rate fx ON fx.rate_date = CAST(t.tx_datetime AS date) AND fx.currency = t.currency
GROUP BY DATEFROMPARTS(YEAR(t.tx_datetime), MONTH(t.tx_datetime), 1)
ORDER BY 1;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`DATE_TRUNC('month', ts)::date`	`DATEFROMPARTS(YEAR(ts), MONTH(ts), 1)`	Pierwszy dzień miesiąca. W SQL Server 2022 można też CAST(DATETRUNC(month, ts) AS date).
`GROUP BY 1`	`GROUP BY <całe wyrażenie>`	Brak grupowania po numerze kolumny. ORDER BY 1 działa w obu systemach.
`FILTER / AVG(::int)`	`SUM(CASE ...) / AVG(CAST AS float)`	Liczenie fraudów.

Funkcje i metody użyte w tym zapytaniu

- **DATE_TRUNC('month' / 'week', ts)**: Obcina datę do początku miesiąca lub tygodnia (tydzień od poniedziałku). SQL Server 2022: DATETRUNC(month, ts) / DATETRUNC(iso_week, ts). Starsze wersje: DATEFROMPARTS(YEAR(ts), MONTH(ts), 1) dla miesiąca.
- **::date vs CAST(... AS date)**: Zamiana typu. W PostgreSQL skrót x::date (tu: obcina godzinę z daty-czasu). W SQL Server CAST(x AS date) lub CONVERT(date, x). Zapis CAST działa też w PostgreSQL.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AVG(is_fraud::int) vs AVG(CAST(is_fraud AS float))**: Średnia z 0/1 to udział jedynek, czyli fraud rate. W PostgreSQL zamieniamy BOOLEAN na liczbę (::int). W SQL Server BIT trzeba zamienić na float, bo AVG z liczby całkowitej dałoby wynik całkowity (0).
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **SUM(x)**: Suma wartości w grupie.
- **JOIN ... ON**: Łączy wiersze z dwóch tabel według warunku, np. transakcja z kursem waluty z tego samego dnia. Zostają tylko wiersze, które mają parę (INNER JOIN).
- **GROUP BY 1 (numer kolumny)**: W PostgreSQL można grupować po numerze kolumny z SELECT-a. SQL Server na to nie pozwala: trzeba powtórzyć całe wyrażenie (np. cały CASE) albo policzyć je wcześniej w podzapytaniu / CROSS APPLY i grupować po aliasie.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

2. Trend tygodniowy (tydzień od poniedziałku)

Co robi: To samo co trend miesięczny, ale data jest obcinana do poniedziałku danego tygodnia. Pokazuje krótsze wahania.

Wynik na Twoich danych: Około 26 tygodni; fraud rate tydzień do tygodnia waha się wokół 1,6%.

PostgreSQL (Twój skrypt)

```
SELECT
    DATE_TRUNC('week', t.tx_datetime)::date            AS week_start,
    COUNT(*)                                           AS transactions,
    COUNT(*) FILTER (WHERE t.is_fraud)                 AS fraud_transactions,
    ROUND(100.0 * AVG(t.is_fraud::int), 2)             AS fraud_rate_pct
FROM fact_transaction t
GROUP BY 1
ORDER BY 1;
```

Microsoft SQL Server (SSMS)

```
-- SQL Server 2022+: DATETRUNC(iso_week, ...) = Monday of the week
SELECT
    CAST(DATETRUNC(iso_week, t.tx_datetime) AS date)   AS week_start,
    COUNT(*)                                           AS transactions,
    SUM(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END)    AS fraud_transactions,
    ROUND(100.0 * AVG(CAST(t.is_fraud AS float)), 2)   AS fraud_rate_pct
FROM fact_transaction t
GROUP BY CAST(DATETRUNC(iso_week, t.tx_datetime) AS date)
ORDER BY 1;

-- Older SQL Server: Monday of the week, independent of @@DATEFIRST
-- DATEADD(day, -((DATEPART(weekday, t.tx_datetime) + @@DATEFIRST - 2) % 7),
--         CAST(t.tx_datetime AS date))
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`DATE_TRUNC('week', ts)`	`DATETRUNC(iso_week, ts) (SQL Server 2022+)`	Poniedziałek tygodnia. Zwykłe week w SQL Server zaczyna tydzień od niedzieli (ustawienia USA), dlatego iso_week.
`(brak)`	`wzór z DATEPART i @@DATEFIRST`	Dla starszych wersji: cofnięcie daty do poniedziałku niezależnie od ustawień serwera.
`GROUP BY 1`	`GROUP BY <całe wyrażenie>`	Brak grupowania po numerze kolumny.

Funkcje i metody użyte w tym zapytaniu

- **DATE_TRUNC('month' / 'week', ts)**: Obcina datę do początku miesiąca lub tygodnia (tydzień od poniedziałku). SQL Server 2022: DATETRUNC(month, ts) / DATETRUNC(iso_week, ts). Starsze wersje: DATEFROMPARTS(YEAR(ts), MONTH(ts), 1) dla miesiąca.
- **DATEPART(weekday, ts) i @@DATEFIRST**: DATEPART(weekday) zwraca numer dnia tygodnia, ale zależny od ustawienia @@DATEFIRST (dla USA tydzień zaczyna się w niedzielę). Wzór DATEADD(day, -((DATEPART(weekday, d) + @@DATEFIRST - 2) % 7), d) cofa datę do poniedziałku niezależnie od tego ustawienia.
- **::date vs CAST(... AS date)**: Zamiana typu. W PostgreSQL skrót x::date (tu: obcina godzinę z daty-czasu). W SQL Server CAST(x AS date) lub CONVERT(date, x). Zapis CAST działa też w PostgreSQL.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AVG(is_fraud::int) vs AVG(CAST(is_fraud AS float))**: Średnia z 0/1 to udział jedynek, czyli fraud rate. W PostgreSQL zamieniamy BOOLEAN na liczbę (::int). W SQL Server BIT trzeba zamienić na float, bo AVG z liczby całkowitej dałoby wynik całkowity (0).
- **GROUP BY 1 (numer kolumny)**: W PostgreSQL można grupować po numerze kolumny z SELECT-a. SQL Server na to nie pozwala: trzeba powtórzyć całe wyrażenie (np. cały CASE) albo policzyć je wcześniej w podzapytaniu / CROSS APPLY i grupować po aliasie.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

3. Trend dzienny + średnia krocząca z 7 dni

Co robi: Krok daily liczy transakcje i fraudy dla każdego dnia. Potem funkcja okna AVG(...) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) liczy średnią z bieżącego dnia i 6 poprzednich. Średnia krocząca wygładza przypadkowe skoki i lepiej pokazuje kierunek trendu.

Wynik na Twoich danych: 181 dni; średnio ok. 23 fraudy dziennie.

PostgreSQL (Twój skrypt)

```
WITH daily AS (
    SELECT
        t.tx_datetime::date                          AS day,
        COUNT(*)                                     AS transactions,
        COUNT(*) FILTER (WHERE t.is_fraud)           AS fraud_transactions
    FROM fact_transaction t
    GROUP BY 1
)
SELECT
    day,
    transactions,
    fraud_transactions,
    ROUND(100.0 * fraud_transactions / transactions, 2) AS fraud_rate_pct,
    ROUND(AVG(fraud_transactions) OVER (ORDER BY day
          ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), 1) AS fraud_7day_avg
FROM daily
ORDER BY day;
```

Microsoft SQL Server (SSMS)

```
WITH daily AS (
    SELECT
        CAST(t.tx_datetime AS date)                  AS day,
        COUNT(*)                                     AS transactions,
        SUM(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END) AS fraud_transactions
    FROM fact_transaction t
    GROUP BY CAST(t.tx_datetime AS date)
)
SELECT
    day,
    transactions,
    fraud_transactions,
    ROUND(100.0 * fraud_transactions / transactions, 2) AS fraud_rate_pct,
    ROUND(AVG(1.0 * fraud_transactions) OVER (ORDER BY day
          ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), 1) AS fraud_7day_avg
FROM daily
ORDER BY day;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`AVG(fraud_transactions)`	`AVG(1.0 * fraud_transactions)`	Pułapka T-SQL: AVG z liczby całkowitej zwraca liczbę całkowitą (np. 23 zamiast 23,4). Mnożenie przez 1.0 daje wynik z ułamkiem. W PostgreSQL AVG z liczb całkowitych zwraca numeric.
`::date, GROUP BY 1`	`CAST(... AS date), GROUP BY <wyrażenie>`	Jak wcześniej.
`ROWS BETWEEN 6 PRECEDING...`	`bez zmian`	Ramka okna w wierszach działa identycznie.

Funkcje i metody użyte w tym zapytaniu

- **WITH nazwa AS (...) (CTE)**: „Tymczasowa tabela na czas jednego zapytania". Dzielimy duże zapytanie na nazwane kroki, a każdy kolejny krok może korzystać z poprzednich. Działa tak samo w obu systemach.
- **::date vs CAST(... AS date)**: Zamiana typu. W PostgreSQL skrót x::date (tu: obcina godzinę z daty-czasu). W SQL Server CAST(x AS date) lub CONVERT(date, x). Zapis CAST działa też w PostgreSQL.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **GROUP BY 1 (numer kolumny)**: W PostgreSQL można grupować po numerze kolumny z SELECT-a. SQL Server na to nie pozwala: trzeba powtórzyć całe wyrażenie (np. cały CASE) albo policzyć je wcześniej w podzapytaniu / CROSS APPLY i grupować po aliasie.
- **100.0 * a / b (dzielenie z ułamkiem)**: Dzielenie dwóch liczb całkowitych daje w obu systemach wynik całkowity (7/2 = 3). Mnożenie przez 100.0 lub 1.0 zamienia liczbę na dziesiętną i wynik ma ułamek.

- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **Funkcja okna: ... OVER (PARTITION BY ... ORDER BY ...)**: Liczy wartość dla każdego wiersza na podstawie „okna" innych wierszy, ale nie skleja wierszy w grupy (w przeciwieństwie do GROUP BY). PARTITION BY dzieli dane na osobne okna (np. osobno dla każdego klienta), ORDER BY ustala kolejność w oknie. Składnia jest taka sama w obu systemach.
- **ROWS BETWEEN ... PRECEDING AND ...**: Ramka okna liczona w wierszach. UNBOUNDED PRECEDING AND 1 PRECEDING = wszystkie wcześniejsze wiersze bez bieżącego (średnia „przed" transakcją, bez podglądania wyniku). 6 PRECEDING AND CURRENT ROW = 7 ostatnich dni (średnia krocząca). Tak samo w obu systemach.
- **AVG(x)**: Średnia arytmetyczna. Uwaga w SQL Server: AVG z kolumny całkowitej (INT, wynik COUNT) zwraca liczbę całkowitą z obciętym ułamkiem (np. 25 zamiast 25,8). Dlatego w T-SQL mnożymy przez 1.0 albo rzutujemy na float.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

4. Pora dnia: o której godzinie najwięcej fraudu?

Co robi: Wyciąga godzinę (0–23) z daty-czasu transakcji, grupuje po niej i liczy fraud rate dla każdej godziny.

Wynik na Twoich danych: Najwyższy fraud rate o 3:00 w nocy; stąd sygnał „nocna godzina 0–4".

PostgreSQL (Twój skrypt)

```
SELECT
    EXTRACT(HOUR FROM t.tx_datetime)::int              AS hour,
    COUNT(*)                                           AS transactions,
    COUNT(*) FILTER (WHERE t.is_fraud)                 AS fraud_transactions,
    ROUND(100.0 * AVG(t.is_fraud::int), 2)             AS fraud_rate_pct
FROM fact_transaction t
GROUP BY 1
ORDER BY 1;
```

Microsoft SQL Server (SSMS)

```
SELECT
    DATEPART(hour, t.tx_datetime)                      AS hour,
    COUNT(*)                                           AS transactions,
    SUM(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END)    AS fraud_transactions,
    ROUND(100.0 * AVG(CAST(t.is_fraud AS float)), 2)   AS fraud_rate_pct
FROM fact_transaction t
GROUP BY DATEPART(hour, t.tx_datetime)
ORDER BY 1;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`EXTRACT(HOUR FROM ts)::int`	`DATEPART(hour, ts)`	Godzina z daty. DATEPART zwraca od razu liczbę całkowitą, EXTRACT w PostgreSQL zwraca liczbę dziesiętną, stąd ::int.
`GROUP BY 1`	`GROUP BY DATEPART(hour, ts)`	Brak grupowania po numerze kolumny.

Funkcje i metody użyte w tym zapytaniu

- **EXTRACT(HOUR FROM ts) vs DATEPART(hour, ts)**: Wyciąga część daty, tu godzinę 0–23.
- **::typ vs CAST(... AS typ)**: Zamiana typu, np. ::numeric, ::int, ::text. W SQL Server CAST(x AS typ). Tekstowy odpowiednik ::text to CAST(x AS VARCHAR(10)).
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AVG(is_fraud::int) vs AVG(CAST(is_fraud AS float))**: Średnia z 0/1 to udział jedynek, czyli fraud rate. W PostgreSQL zamieniamy BOOLEAN na liczbę (::int). W SQL Server BIT trzeba zamienić na float, bo AVG z liczby całkowitej dałoby wynik całkowity (0).
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **GROUP BY 1 (numer kolumny)**: W PostgreSQL można grupować po numerze kolumny z SELECT-a. SQL Server na to nie pozwala: trzeba powtórzyć całe wyrażenie (np. cały CASE) albo policzyć je wcześniej w podzapytaniu / CROSS APPLY i grupować po aliasie.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

02_sql_analysis/03_customer_behaviour.sql

1. Top 20 klientów według wartości transakcji

Co robi: Sumuje wartość transakcji każdego klienta w EUR, dokłada kraj i plan konta, i pokazuje 20 największych. Sprawdzamy, czy najwięksi klienci to zwykli klienci, czy konta z fraudem.

Wynik na Twoich danych: 20 wierszy; część największych kont ma transakcje fraudowe (oszuści wydają dużo naraz).

PostgreSQL (Twój skrypt)

```
SELECT
    t.customer_id,
    c.country,
    c.plan,
    COUNT(*)                                            AS transactions,
    ROUND(SUM(t.amount * fx.eur_rate), 0)               AS value_eur,
    ROUND(AVG(t.amount * fx.eur_rate), 2)               AS avg_transaction_eur,
    COUNT(*) FILTER (WHERE t.is_fraud)                  AS fraud_transactions
FROM fact_transaction t
JOIN dim_customer c USING (customer_id)
JOIN dim_fx_rate fx ON fx.rate_date = t.tx_datetime::date AND fx.currency = t.currency
GROUP BY t.customer_id, c.country, c.plan
ORDER BY value_eur DESC
LIMIT 20;
```

Microsoft SQL Server (SSMS)

```
SELECT TOP 20
    t.customer_id,
    c.country,
    c.[plan],
    COUNT(*)                                            AS transactions,
    ROUND(SUM(t.amount * fx.eur_rate), 0)               AS value_eur,
    ROUND(AVG(t.amount * fx.eur_rate), 2)               AS avg_transaction_eur,
    SUM(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END)     AS fraud_transactions
FROM fact_transaction t
JOIN dim_customer c ON c.customer_id = t.customer_id
JOIN dim_fx_rate fx ON fx.rate_date = CAST(t.tx_datetime AS date) AND fx.currency = t.currency
GROUP BY t.customer_id, c.country, c.[plan]
ORDER BY value_eur DESC;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`LIMIT 20`	`SELECT TOP 20`	Ograniczenie liczby wierszy.
`plan`	`[plan]`	Słowo zarezerwowane w T-SQL.
`USING / ::date / FILTER`	`ON / CAST / SUM(CASE)`	Jak wcześniej.

Funkcje i metody użyte w tym zapytaniu

- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **SUM(x)**: Suma wartości w grupie.
- **AVG(x)**: Średnia arytmetyczna. Uwaga w SQL Server: AVG z kolumny całkowitej (INT, wynik COUNT) zwraca liczbę całkowitą z obciętym ułamkiem (np. 25 zamiast 25,8). Dlatego w T-SQL mnożymy przez 1.0 albo rzutujemy na float.
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **JOIN ... ON**: Łączy wiersze z dwóch tabel według warunku, np. transakcja z kursem waluty z tego samego dnia. Zostają tylko wiersze, które mają parę (INNER JOIN).
- **JOIN ... USING (kolumna)**: Skrót w PostgreSQL: gdy obie tabele mają kolumnę o tej samej nazwie, USING (customer_id) znaczy ON a.customer_id = b.customer_id. SQL Server nie zna USING, więc trzeba pisać pełne ON.
- **::date vs CAST(... AS date)**: Zamiana typu. W PostgreSQL skrót x::date (tu: obcina godzinę z daty-czasu). W SQL Server CAST(x AS date) lub CONVERT(date, x). Zapis CAST działa też w PostgreSQL.
- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).
- **LIMIT n vs TOP n**: Zwraca tylko pierwsze n wierszy (po sortowaniu). PostgreSQL: LIMIT n na końcu zapytania. SQL Server: SELECT TOP n tuż po SELECT.

2. Top 20 klientów według liczby transakcji

Co robi: Dla każdego klienta liczy transakcje, liczbę dni, w których był aktywny (COUNT DISTINCT daty), i średnią liczbę transakcji na aktywny dzień. Bardzo wysoka liczba transakcji dziennie może wskazywać na card testing.

Wynik na Twoich danych: 20 najaktywniejszych klientów i ich tempo płatności.

PostgreSQL (Twój skrypt)

```
SELECT
    t.customer_id,
    COUNT(*)                                          AS transactions,
    COUNT(DISTINCT t.tx_datetime::date)               AS active_days,
    ROUND(COUNT(*)::numeric / COUNT(DISTINCT t.tx_datetime::date), 1) AS tx_per_active_day,
    COUNT(*) FILTER (WHERE t.is_fraud)                AS fraud_transactions
FROM fact_transaction t
GROUP BY t.customer_id
ORDER BY transactions DESC
LIMIT 20;
```

Microsoft SQL Server (SSMS)

```
SELECT TOP 20
    t.customer_id,
    COUNT(*)                                          AS transactions,
    COUNT(DISTINCT CAST(t.tx_datetime AS date))       AS active_days,
    ROUND(1.0 * COUNT(*) / COUNT(DISTINCT CAST(t.tx_datetime AS date)), 1) AS tx_per_active_day,
    SUM(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END)   AS fraud_transactions
FROM fact_transaction t
GROUP BY t.customer_id
ORDER BY transactions DESC;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`COUNT(*)::numeric`	`1.0 * COUNT(*)`	Wymuszenie dzielenia z ułamkiem (inaczej 7/2 = 3).
`LIMIT 20`	`SELECT TOP 20`	Ograniczenie liczby wierszy.
`::date / FILTER`	`CAST / SUM(CASE)`	Jak wcześniej.

Funkcje i metody użyte w tym zapytaniu

- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(DISTINCT x)**: Liczy różne wartości, np. ile różnych urządzeń albo dni. COUNT(*) − COUNT(DISTINCT id) pokazuje liczbę duplikatów.
- **::date vs CAST(... AS date)**: Zamiana typu. W PostgreSQL skrót x::date (tu: obcina godzinę z daty-czasu). W SQL Server CAST(x AS date) lub CONVERT(date, x). Zapis CAST działa też w PostgreSQL.
- **::typ vs CAST(... AS typ)**: Zamiana typu, np. ::numeric, ::int, ::text. W SQL Server CAST(x AS typ). Tekstowy odpowiednik ::text to CAST(x AS VARCHAR(10)).
- **100.0 * a / b (dzielenie z ułamkiem)**: Dzielenie dwóch liczb całkowitych daje w obu systemach wynik całkowity (7/2 = 3). Mnożenie przez 100.0 lub 1.0 zamienia liczbę na dziesiętną i wynik ma ułamek.
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).
- **LIMIT n vs TOP n**: Zwraca tylko pierwsze n wierszy (po sortowaniu). PostgreSQL: LIMIT n na końcu zapytania. SQL Server: SELECT TOP n tuż po SELECT.

3. Średnia transakcja: klienci z fraudem vs bez fraudu

Co robi: Krok per_customer liczy dla każdego klienta średnią kwotę, liczbę transakcji i to, czy miał choć jeden fraud (BOOL_OR). Potem dzielimy klientów na dwie grupy i porównujemy średnie oraz medianę.

Wynik na Twoich danych: 849 klientów z fraudem: średnia transakcja 255,69 EUR; klienci bez fraudu: 83,15 EUR.

PostgreSQL (Twój skrypt)

```
WITH per_customer AS (
    SELECT
        t.customer_id,
        AVG(t.amount * fx.eur_rate)                    AS avg_eur,
        COUNT(*)                                       AS transactions,
        BOOL_OR(t.is_fraud)                            AS had_fraud      -- true if at least one transaction was
fraud
    FROM fact_transaction t
    JOIN dim_fx_rate fx ON fx.rate_date = t.tx_datetime::date AND fx.currency = t.currency
    GROUP BY t.customer_id
)
SELECT
    CASE WHEN had_fraud THEN 'customer with fraud' ELSE 'customer without fraud' END AS customer_group,
    COUNT(*)                                           AS customers,
    ROUND(AVG(transactions), 1)                        AS avg_transactions,
    ROUND(AVG(avg_eur), 2)                             AS avg_transaction_eur,
    ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY avg_eur)::numeric, 2) AS median_avg_eur
FROM per_customer
GROUP BY 1;
```

Microsoft SQL Server (SSMS)

```
WITH per_customer AS (
    SELECT
        t.customer_id,
        AVG(t.amount * fx.eur_rate)                    AS avg_eur,
        COUNT(*)                                       AS transactions,
        MAX(CAST(t.is_fraud AS int))                   AS had_fraud      -- 1 if at least one transaction was fraud
    FROM fact_transaction t
    JOIN dim_fx_rate fx ON fx.rate_date = CAST(t.tx_datetime AS date) AND fx.currency = t.currency
    GROUP BY t.customer_id
),
with_median AS (
   -- the median must be a window function in T-SQL
   SELECT *,
          PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY avg_eur)
              OVER (PARTITION BY had_fraud) AS median_avg_eur
   FROM per_customer
)
SELECT
    CASE WHEN had_fraud = 1 THEN 'customer with fraud' ELSE 'customer without fraud' END AS customer_group,
    COUNT(*)                                           AS customers,
    ROUND(AVG(1.0 * transactions), 1)                  AS avg_transactions,
    ROUND(AVG(avg_eur), 2)                             AS avg_transaction_eur,
    ROUND(MAX(median_avg_eur), 2)                      AS median_avg_eur
FROM with_median
GROUP BY had_fraud;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`BOOL_OR(is_fraud)`	`MAX(CAST(is_fraud AS int))`	„Czy był choć jeden fraud”: maksimum z 0/1 to 1, jeśli choć jedna transakcja była fraudem.
`PERCENTILE_CONT` jako agregat	osobny krok `with_median` z `OVER (PARTITION BY had_fraud)`, potem `MAX(...)`	Mediana w T-SQL jest funkcją okna, więc liczymy ją w osobnym kroku; w każdej grupie ma tę samą wartość, a MAX tylko ją „wyciąga”.
`AVG(transactions)`	`AVG(1.0 * transactions)`	AVG z liczby całkowitej w T-SQL obcina ułamek.
`GROUP BY 1`	`GROUP BY had_fraud`	Grupujemy po kolumnie, z której powstaje CASE: wynik jest ten sam.

Funkcje i metody użyte w tym zapytaniu

- **WITH nazwa AS (...) (CTE)**: „Tymczasowa tabela na czas jednego zapytania”. Dzielimy duże zapytanie na nazwane kroki, a każdy kolejny krok może korzystać z poprzednich. Działa tak samo w obu systemach.
- **AVG(x)**: Średnia arytmetyczna. Uwaga w SQL Server: AVG z kolumny całkowitej (INT, wynik COUNT) zwraca liczbę całkowitą z obciętym ułamkiem (np. 25 zamiast 25,8). Dlatego w T-SQL mnożymy przez 1.0 albo rzutujemy na float.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **BOOL_OR(x) vs MAX(CAST(x AS int))**: Czy choć jedna wartość w grupie jest prawdą (np. czy klient miał choć jeden fraud). SQL Server nie ma BOOL_OR; MAX z 0/1 daje ten sam efekt (1 = tak).

- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli”: dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **PERCENTILE_CONT(p) WITHIN GROUP (ORDER BY x)**: Percentyl z interpolacją: 0.5 to mediana, 0.999 to wartość, poniżej której jest 99,9% danych. W PostgreSQL działa jak zwykła funkcja agregująca (z GROUP BY). W SQL Server tylko jako funkcja okna: trzeba dopisać OVER (PARTITION BY ...) lub OVER () i usunąć powtórzenia (DISTINCT / TOP 1).
- **Funkcja okna: ... OVER (PARTITION BY ... ORDER BY ...)**: Liczy wartość dla każdego wiersza na podstawie „okna” innych wierszy, ale nie skleja wierszy w grupy (w przeciwieństwie do GROUP BY). PARTITION BY dzieli dane na osobne okna (np. osobno dla każdego klienta), ORDER BY ustala kolejność w oknie. Składnia jest taka sama w obu systemach.
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **::typ vs CAST(... AS typ)**: Zamiana typu, np. ::numeric, ::int, ::text. W SQL Server CAST(x AS typ). Tekstowy odpowiednik ::text to CAST(x AS VARCHAR(10)).
- **GROUP BY 1 (numer kolumny)**: W PostgreSQL można grupować po numerze kolumny z SELECT-a. SQL Server na to nie pozwala: trzeba powtórzyć całe wyrażenie (np. cały CASE) albo policzyć je wcześniej w podzapytaniu / CROSS APPLY i grupować po aliasie.
- **MIN(x) / MAX(x)**: Najmniejsza / największa wartość w grupie (np. najwyższy wynik ryzyka klienta).

4. Liczba urządzeń klienta a ryzyko

Co robi: Dla każdego klienta liczy, z ilu różnych urządzeń płacił, i czy miał fraud. Potem grupuje klientów według liczby urządzeń (1, 2, 3, 4+) i pokazuje, jaki procent z nich miał fraud.

Wynik na Twoich danych: 1 urządzenie: 5,81% klientów z fraudem; 3 urządzenia: 18,98%. Więcej urządzeń = większe ryzyko przejęcia konta.

PostgreSQL (Twój skrypt)

```
WITH per_customer AS (
    SELECT
        customer_id,
        COUNT(DISTINCT device_id)                       AS devices,
        BOOL_OR(is_fraud)                               AS had_fraud
    FROM fact_transaction
    GROUP BY customer_id
)
SELECT
    CASE WHEN devices >= 4 THEN '4+' ELSE devices::text END AS devices_used,
    COUNT(*)                                            AS customers,
    COUNT(*) FILTER (WHERE had_fraud)                   AS customers_with_fraud,
    ROUND(100.0 * AVG(had_fraud::int), 2)               AS pct_customers_with_fraud
FROM per_customer
GROUP BY 1
ORDER BY 1;
```

Microsoft SQL Server (SSMS)

```
WITH per_customer AS (
    SELECT
        customer_id,
        COUNT(DISTINCT device_id)                       AS devices,
        MAX(CAST(is_fraud AS int))                      AS had_fraud
    FROM fact_transaction
    GROUP BY customer_id
)
SELECT
    x.devices_used,
    COUNT(*)                                            AS customers,
    SUM(had_fraud)                                      AS customers_with_fraud,
    ROUND(100.0 * AVG(CAST(had_fraud AS float)), 2)     AS pct_customers_with_fraud
FROM per_customer
CROSS APPLY (SELECT CASE WHEN devices >= 4 THEN '4+'
                         ELSE CAST(devices AS VARCHAR(10)) END AS devices_used) x
GROUP BY x.devices_used
ORDER BY 1;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`BOOL_OR`	`MAX(CAST(... AS int))`	Czy klient miał choć jeden fraud (1/0).
`devices::text`	`CAST(devices AS VARCHAR(10))`	Liczba zamieniona na tekst, żeby pasowała do '4+' w tej samej kolumnie.
`GROUP BY 1`	`CROSS APPLY (SELECT CASE ... AS devices_used) x + GROUP BY x.devices_used`	Sztuczka T-SQL: liczymy wyrażenie raz w CROSS APPLY, nadajemy mu nazwę i po niej grupujemy, zamiast przepisywać cały CASE.
`COUNT(*) FILTER (WHERE had_fraud)`	`SUM(had_fraud)`	had_fraud to już 0/1, więc suma = liczba klientów z fraudem.

Funkcje i metody użyte w tym zapytaniu

- **WITH nazwa AS (...) (CTE)**: „Tymczasowa tabela na czas jednego zapytania”. Dzielimy duże zapytanie na nazwane kroki, a każdy kolejny krok może korzystać z poprzednich. Działa tak samo w obu systemach.
- **COUNT(DISTINCT x)**: Liczy różne wartości, np. ile różnych urządzeń albo dni. COUNT(*) − COUNT(DISTINCT id) pokazuje liczbę duplikatów.
- **BOOL_OR(x) vs MAX(CAST(x AS int))**: Czy choć jedna wartość w grupie jest prawdą (np. czy klient miał choć jeden fraud). SQL Server nie ma BOOL_OR; MAX z 0/1 daje ten sam efekt (1 = tak).
- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli”: dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **::typ vs CAST(... AS typ)**: Zamiana typu, np. ::numeric, ::int, ::text. W SQL Server CAST(x AS typ). Tekstowy odpowiednik ::text to CAST(x AS VARCHAR(10)).
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AVG(is_fraud::int) vs AVG(CAST(is_fraud AS float))**: Średnia z 0/1 to udział jedynek, czyli fraud rate. W PostgreSQL zamieniamy BOOLEAN na liczbę (::int). W SQL Server BIT trzeba zamienić na float, bo AVG z liczby całkowitej dałoby wynik całkowity (0).
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **GROUP BY 1 (numer kolumny)**: W PostgreSQL można grupować po numerze kolumny z SELECT-a. SQL Server na to nie pozwala: trzeba powtórzyć całe wyrażenie (np. cały CASE) albo policzyć je wcześniej w podzapytaniu / CROSS APPLY i grupować po aliasie.
- **OUTER APPLY (podzapytanie)**: T-SQL: dla każdego wiersza uruchamia podzapytanie, które może korzystać z jego kolumn (tu: policz transakcje tego klienta z ostatnich 10 minut). OUTER zostawia wiersz, nawet gdy podzapytanie nic nie zwróci. Odpowiednik w PostgreSQL: LEFT JOIN LATERAL.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

02_sql_analysis/04_window_functions.sql

1. ROW_NUMBER i LAG: historia jednego klienta krok po kroku

Co robi: Pokazuje wszystkie transakcje klienta 8080 po kolei. ROW_NUMBER numeruje je (1 = pierwsza), a LAG dopisuje do każdej transakcji dane z poprzedniej: kraj, urządzenie i czas, jaki upłynął. Dzięki temu widać na oko zmianę kraju, nowe urządzenie albo serię szybkich płatności.

Wynik na Twoich danych: Historia klienta 8080 z kolumną time_since_prev (np. 00:03:12 = 3 min 12 s od poprzedniej transakcji).

PostgreSQL (Twój skrypt)

```
SELECT
    t.customer_id,
    ROW_NUMBER() OVER w                                     AS tx_no,
    t.tx_datetime,
    t.country,
    LAG(t.country) OVER w                                   AS prev_country,
    t.tx_datetime - LAG(t.tx_datetime) OVER w               AS time_since_prev,
    t.device_id,
    LAG(t.device_id) OVER w                                 AS prev_device,
    t.amount, t.currency, t.is_fraud
FROM fact_transaction t
WHERE t.customer_id = 8080
WINDOW w AS (PARTITION BY t.customer_id ORDER BY t.tx_datetime, t.transaction_id)   -- one window definition, used
many times (T-SQL: SQL Server 2022+)
ORDER BY tx_no;
```

Microsoft SQL Server (SSMS)

```
SELECT
    t.customer_id,
    ROW_NUMBER() OVER w                                     AS tx_no,
    t.tx_datetime,
    t.country,
    LAG(t.country) OVER w                                   AS prev_country,
    DATEDIFF(minute, LAG(t.tx_datetime) OVER w, t.tx_datetime) AS minutes_since_prev,
    t.device_id,
    LAG(t.device_id) OVER w                                 AS prev_device,
    t.amount, t.currency, t.is_fraud
FROM fact_transaction t
WHERE t.customer_id = 8080
WINDOW w AS (PARTITION BY t.customer_id ORDER BY t.tx_datetime, t.transaction_id)   -- SQL Server 2022+ only
ORDER BY tx_no;

-- Older SQL Server: no WINDOW clause, repeat the window in every OVER, e.g.
-- LAG(t.country) OVER (PARTITION BY t.customer_id ORDER BY t.tx_datetime, t.transaction_id)
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`ts − LAG(ts) (wynik: INTERVAL)`	`DATEDIFF(minute, LAG(ts), ts) (wynik: liczba minut)`	SQL Server nie odejmuje dat; DATEDIFF podaje różnicę w wybranej jednostce. Dlatego zmieniona nazwa kolumny: minutes_since_prev.
`WINDOW w AS (...)`	`to samo, ale tylko SQL Server 2022+`	W starszych wersjach trzeba wpisać pełne OVER (...) przy każdej funkcji.

Funkcje i metody użyte w tym zapytaniu

- **ROW_NUMBER()**: Numeruje wiersze w oknie 1, 2, 3... według ORDER BY. Tu: która to transakcja klienta (1 = pierwsza) albo pierwsza transakcja w danym kraju.
- **LAG(x)**: Wartość z poprzedniego wiersza w oknie, np. kraj lub urządzenie z poprzedniej transakcji klienta. Pierwsza transakcja dostaje NULL.
- **Funkcja okna: ... OVER (PARTITION BY ... ORDER BY ...)**: Liczy wartość dla każdego wiersza na podstawie „okna" innych wierszy, ale nie skleja wierszy w grupy (w przeciwieństwie do GROUP BY). PARTITION BY dzieli dane na osobne okna (np. osobno dla każdego klienta), ORDER BY ustala kolejność w oknie. Składnia jest taka sama w obu systemach.
- **WINDOW w AS (...)**: Nazwana definicja okna, której używamy wiele razy (OVER w). Skraca zapis. W SQL Server dostępna dopiero od wersji 2022; w starszych trzeba powtórzyć OVER (PARTITION BY ... ORDER BY ...) przy każdej funkcji.
- **ts − LAG(ts) vs DATEDIFF**: W PostgreSQL odjęcie dwóch dat-czasów daje INTERVAL (np. 00:12:30). SQL Server tak nie potrafi: DATEDIFF(minute, a, b) zwraca liczbę pełnych minut między a i b.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

2. Sygnały ryzyka dla każdej transakcji i ich skuteczność

Co robi: Najważniejsze zapytanie analityczne. (1) tx: transakcja z kategorią i kwotą w EUR. (2) features: funkcje okna liczą kontekst każdej transakcji: która to transakcja klienta, średnia jego WCZEŚNIEJSZYCH kwot, ile transakcji zrobił w ostatnich 10 minutach, kiedy pierwszy raz użył urządzenia, czy to pierwsza transakcja w danym kraju. (3) signals: CASE zamienia to na 6 sygnałów 0/1. (4) Na końcu CROSS JOIN LATERAL obraca 6 kolumn sygnałów w wiersze i dla każdego sygnału liczymy: w ilu transakcjach się zapalił, fraud rate, gdy włączony i gdy wyłączony, oraz jaki procent wszystkich fraudów łapie.

Wynik na Twoich danych: 5+ tx w 10 min: 100% (efekt danych); nowe urządzenie: 43,45% vs 0,58%, łapie 65,8% fraudów; ryzykowna kategoria: 11,59%, łapie 80,7%; noc: 3,04% vs 1,61%.

PostgreSQL (Twój skrypt)

```
WITH tx AS (
    SELECT
        t.transaction_id, t.customer_id, t.tx_datetime, t.country, t.device_id, t.is_fraud,
        m.merchant_category,
        t.amount * fx.eur_rate                                  AS amount_eur
    FROM fact_transaction t
    JOIN dim_merchant m USING (merchant_id)
    JOIN dim_fx_rate fx ON fx.rate_date = t.tx_datetime::date AND fx.currency = t.currency
),
features AS (
    SELECT
        tx.*,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY tx_datetime, transaction_id)  AS tx_no,
        -- customer's historical average: only EARLIER transactions (excluding the current one, so we don't "peek" at
the result)
        AVG(amount_eur) OVER (PARTITION BY customer_id ORDER BY tx_datetime, transaction_id
                              ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING)             AS hist_avg_eur,
        -- velocity: how many transactions the customer made in the last 10 minutes (including the current one)
        COUNT(*) OVER (PARTITION BY customer_id ORDER BY tx_datetime
                       RANGE BETWEEN INTERVAL '10 minutes' PRECEDING AND CURRENT ROW)       AS tx_last_10_min,
        -- rolling volume: sum of the customer's amounts in the last 24 hours
        SUM(amount_eur) OVER (PARTITION BY customer_id ORDER BY tx_datetime
                              RANGE BETWEEN INTERVAL '24 hours' PRECEDING AND CURRENT ROW)  AS sum_24h_eur,
        -- when the customer first used this device, and when they made their first transaction at all
        MIN(tx_datetime) OVER (PARTITION BY customer_id, device_id)                         AS device_first_use,
        MIN(tx_datetime) OVER (PARTITION BY customer_id)                                    AS customer_first_tx,
        -- customer's first transaction in a given country
        ROW_NUMBER() OVER (PARTITION BY customer_id, country ORDER BY tx_datetime, transaction_id) AS
tx_no_in_country
    FROM tx
),
signals AS (
    SELECT
        transaction_id, is_fraud,
        -- signals from section 10 of the plan (1 = on, 0 = off)
        CASE WHEN amount_eur > 3 * hist_avg_eur THEN 1 ELSE 0 END                            AS s_amount_3x_avg,
        -- new device = device first used within the last 24 h (and it is not the customer's first device)
        CASE WHEN device_first_use > customer_first_tx
              AND tx_datetime < device_first_use + INTERVAL '24 hours' THEN 1 ELSE 0 END     AS s_new_device,
        CASE WHEN tx_no > 1 AND tx_no_in_country = 1 THEN 1 ELSE 0 END                       AS s_new_country,
        CASE WHEN tx_last_10_min >= 5 THEN 1 ELSE 0 END                                      AS s_5_tx_in_10_min,
        CASE WHEN merchant_category IN ('crypto', 'gift_cards', 'electronics') THEN 1 ELSE 0 END AS s_risky_category,
        CASE WHEN EXTRACT(HOUR FROM tx_datetime) BETWEEN 0 AND 4 THEN 1 ELSE 0 END           AS s_night_hour
    FROM features
)
-- One row per signal: how many transactions have it, their fraud rate, and what share of all frauds it catches
SELECT signal,
       COUNT(*) FILTER (WHERE is_on = 1)                                                     AS tx_with_signal,
       ROUND(100.0 * AVG(is_fraud::int) FILTER (WHERE is_on = 1), 2)                         AS fraud_rate_when_on_pct,
       ROUND(100.0 * AVG(is_fraud::int) FILTER (WHERE is_on = 0), 2)                         AS fraud_rate_when_off_pct,
       ROUND(100.0 * COUNT(*) FILTER (WHERE is_on = 1 AND is_fraud)
                   / COUNT(*) FILTER (WHERE is_fraud), 1)                                    AS fraud_caught_pct
FROM signals
CROSS JOIN LATERAL (VALUES                      -- "unpivots" the signal columns into rows (T-SQL: CROSS APPLY
(VALUES ...))
    ('amount > 3x customer avg', s_amount_3x_avg),
    ('new device',               s_new_device),
    ('new country',              s_new_country),
    ('5+ tx in 10 min',          s_5_tx_in_10_min),
    ('risky category',           s_risky_category),
    ('night hour (0-4)',         s_night_hour)
) AS v(signal, is_on)
GROUP BY signal
ORDER BY fraud_rate_when_on_pct DESC;
```

Microsoft SQL Server (SSMS)

```
-- Step 1: base data into a temp table with an index (needed for the time windows below)
DROP TABLE IF EXISTS #tx;
SELECT
    t.transaction_id, t.customer_id, t.tx_datetime, t.country, t.device_id, t.is_fraud,
    m.merchant_category,
    t.amount * fx.eur_rate                                      AS amount_eur
INTO #tx
FROM fact_transaction t
JOIN dim_merchant m ON m.merchant_id = t.merchant_id
JOIN dim_fx_rate fx ON fx.rate_date = CAST(t.tx_datetime AS date) AND fx.currency = t.currency;

CREATE INDEX ix_tx_customer_time ON #tx (customer_id, tx_datetime);

-- Step 2: features, signals and the summary
WITH features AS (
    SELECT
        tx.*,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY tx_datetime, transaction_id)  AS tx_no,
        AVG(amount_eur) OVER (PARTITION BY customer_id ORDER BY tx_datetime, transaction_id
                              ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING)            AS hist_avg_eur,
        v10.tx_last_10_min,
        v24.sum_24h_eur,
        MIN(tx_datetime) OVER (PARTITION BY customer_id, device_id)                        AS device_first_use,
        MIN(tx_datetime) OVER (PARTITION BY customer_id)                                   AS customer_first_tx,
        ROW_NUMBER() OVER (PARTITION BY customer_id, country ORDER BY tx_datetime, transaction_id) AS
tx_no_in_country
    FROM #tx AS tx
    -- time windows: T-SQL has no RANGE ... INTERVAL, so we count with a subquery per row
    OUTER APPLY (SELECT COUNT(*) AS tx_last_10_min
                 FROM #tx t2
                 WHERE t2.customer_id = tx.customer_id
                   AND t2.tx_datetime BETWEEN DATEADD(minute, -10, tx.tx_datetime) AND tx.tx_datetime) v10
    OUTER APPLY (SELECT SUM(t2.amount_eur) AS sum_24h_eur
                 FROM #tx t2
                 WHERE t2.customer_id = tx.customer_id
                   AND t2.tx_datetime BETWEEN DATEADD(hour, -24, tx.tx_datetime) AND tx.tx_datetime) v24
),
signals AS (
    SELECT
        transaction_id, is_fraud,
        CASE WHEN amount_eur > 3 * hist_avg_eur THEN 1 ELSE 0 END                          AS s_amount_3x_avg,
        CASE WHEN device_first_use > customer_first_tx
              AND tx_datetime < DATEADD(hour, 24, device_first_use) THEN 1 ELSE 0 END      AS s_new_device,
        CASE WHEN tx_no > 1 AND tx_no_in_country = 1 THEN 1 ELSE 0 END                     AS s_new_country,
        CASE WHEN tx_last_10_min >= 5 THEN 1 ELSE 0 END                                    AS s_5_tx_in_10_min,
        CASE WHEN merchant_category IN ('crypto', 'gift_cards', 'electronics') THEN 1 ELSE 0 END AS s_risky_category,
        CASE WHEN DATEPART(hour, tx_datetime) BETWEEN 0 AND 4 THEN 1 ELSE 0 END            AS s_night_hour
    FROM features
)
SELECT signal,
       SUM(CASE WHEN is_on = 1 THEN 1 ELSE 0 END)                                  AS tx_with_signal,
       ROUND(100.0 * AVG(CASE WHEN is_on = 1 THEN CAST(is_fraud AS float) END), 2)  AS fraud_rate_when_on_pct,
       ROUND(100.0 * AVG(CASE WHEN is_on = 0 THEN CAST(is_fraud AS float) END), 2)  AS fraud_rate_when_off_pct,
       ROUND(100.0 * SUM(CASE WHEN is_on = 1 AND is_fraud = 1 THEN 1 ELSE 0 END)
                   / SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END), 1)             AS fraud_caught_pct
FROM signals
CROSS APPLY (VALUES
    ('amount > 3x customer avg', s_amount_3x_avg),
    ('new device',               s_new_device),
    ('new country',              s_new_country),
    ('5+ tx in 10 min',          s_5_tx_in_10_min),
    ('risky category',           s_risky_category),
    ('night hour (0-4)',         s_night_hour)
) AS v(signal, is_on)
GROUP BY signal
ORDER BY fraud_rate_when_on_pct DESC;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`RANGE BETWEEN INTERVAL '10 minutes' PRECEDING`	`OUTER APPLY (SELECT COUNT(*) ... WHERE t2.tx_datetime BETWEEN DATEADD(minute, -10, ts) AND ts)`	Największa różnica w projekcie. SQL Server nie ma okna liczonego w czasie, więc dla każdej transakcji osobne podzapytanie liczy transakcje klienta z ostatnich 10 minut (i sumę z 24 h).
`CTE tx`	`tabela tymczasowa #tx + CREATE INDEX`	Podzapytania w OUTER APPLY przeszukują dane 258 tys. razy; bez indeksu po (customer_id, tx_datetime) trwałoby to bardzo długo.
`ts + INTERVAL '24 hours'`	`DATEADD(hour, 24, ts)`	Przesunięcie daty.
`EXTRACT(HOUR FROM ts)`	`DATEPART(hour, ts)`	Godzina.
`COUNT/AVG ... FILTER (WHERE ...)`	`SUM/AVG(CASE WHEN ... END)`	Liczenie warunkowe.
`CROSS JOIN LATERAL (VALUES ...)`	`CROSS APPLY (VALUES ...)`	Unpivot: kolumny sygnałów zamienione w wiersze.

Funkcje i metody użyte w tym zapytaniu

- **Tabela tymczasowa #nazwa (T-SQL)**: Tabela, która istnieje tylko do końca sesji w SSMS. Zapisujemy do niej wynik pośredni i zakładamy indeks, żeby kolejne kroki (OUTER APPLY) działały szybko. W PostgreSQL odpowiednikiem jest CREATE TEMP TABLE, ale tam nie było potrzebne, bo okno RANGE robi to samo w jednym przebiegu.
- **CREATE INDEX**: Indeks to „spis treści" dla kolumn: przyspiesza wyszukiwanie wierszy (tu: transakcji danego klienta w danym czasie).
- **WITH nazwa AS (...) (CTE)**: „Tymczasowa tabela na czas jednego zapytania". Dzielimy duże zapytanie na nazwane kroki, a każdy kolejny krok może korzystać z poprzednich. Działa tak samo w obu systemach.
- **JOIN ... ON**: Łączy wiersze z dwóch tabel według warunku, np. transakcja z kursem waluty z tego samego dnia. Zostają tylko wiersze, które mają parę (INNER JOIN).
- **JOIN ... USING (kolumna)**: Skrót w PostgreSQL: gdy obie tabele mają kolumnę o tej samej nazwie, USING (customer_id) znaczy ON a.customer_id = b.customer_id. SQL Server nie zna USING, więc trzeba pisać pełne ON.
- **::date vs CAST(... AS date)**: Zamiana typu. W PostgreSQL skrót x::date (tu: obcina godzinę z daty-czasu). W SQL Server CAST(x AS date) lub CONVERT(date, x). Zapis CAST działa też w PostgreSQL.
- **Funkcja okna: ... OVER (PARTITION BY ... ORDER BY ...)**: Liczy wartość dla każdego wiersza na podstawie „okna" innych wierszy, ale nie skleja wierszy w grupy (w przeciwieństwie do GROUP BY). PARTITION BY dzieli dane na osobne okna (np. osobno dla każdego klienta), ORDER BY ustala kolejność w oknie. Składnia jest taka sama w obu systemach.
- **ROW_NUMBER()**: Numeruje wiersze w oknie 1, 2, 3... według ORDER BY. Tu: która to transakcja klienta (1 = pierwsza) albo pierwsza transakcja w danym kraju.
- **ROWS BETWEEN ... PRECEDING AND ...**: Ramka okna liczona w wierszach. UNBOUNDED PRECEDING AND 1 PRECEDING = wszystkie wcześniejsze wiersze bez bieżącego (średnia „przed" transakcją, bez podglądania wyniku). 6 PRECEDING AND CURRENT ROW = 7 ostatnich dni (średnia krocząca). Tak samo w obu systemach.
- **AVG(x)**: Średnia arytmetyczna. Uwaga w SQL Server: AVG z kolumny całkowitej (INT, wynik COUNT) zwraca liczbę całkowitą z obciętym ułamkiem (np. 25 zamiast 25,8). Dlatego w T-SQL mnożymy przez 1.0 albo rzutujemy na float.
- **RANGE BETWEEN INTERVAL '10 minutes' PRECEDING AND CURRENT ROW**: Ramka okna liczona w CZASIE, nie w wierszach: wszystkie transakcje klienta z ostatnich 10 minut. To jedna z największych przewag PostgreSQL. SQL Server tego nie obsługuje, więc liczymy to podzapytaniem w OUTER APPLY z warunkiem na czas (BETWEEN DATEADD(minute, -10, ts) AND ts).
- **OUTER APPLY (podzapytanie)**: T-SQL: dla każdego wiersza uruchamia podzapytanie, które może korzystać z jego kolumn (tu: policz transakcje tego klienta z ostatnich 10 minut). OUTER zostawia wiersz, nawet gdy podzapytanie nic nie zwróci. Odpowiednik w PostgreSQL: LEFT JOIN LATERAL.
- **MIN(x) OVER (PARTITION BY ...)**: Najmniejsza wartość w całym oknie, dopisana do każdego wiersza. Tu: kiedy klient pierwszy raz użył danego urządzenia i kiedy zrobił pierwszą transakcję w ogóle.
- **INTERVAL '24 hours' vs DATEADD**: PostgreSQL ma typ „odcinek czasu": ts + INTERVAL '24 hours' to data przesunięta o dobę. SQL Server nie ma takiego typu; używamy DATEADD(hour, 24, ts) (albo minute, day).
- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli": dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **IN ('a', 'b', ...)**: Czy wartość jest jedną z listy; krótszy zapis wielu OR.
- **EXTRACT(HOUR FROM ts) vs DATEPART(hour, ts)**: Wyciąga część daty, tu godzinę 0–23.
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AVG(is_fraud::int) vs AVG(CAST(is_fraud AS float))**: Średnia z 0/1 to udział jedynek, czyli fraud rate. W PostgreSQL zamieniamy BOOLEAN na liczbę (::int). W SQL Server BIT trzeba zamienić na float, bo AVG z liczby całkowitej dałoby

wynik całkowity (0).

- **100.0 * a / b (dzielenie z ułamkiem)**: Dzielenie dwóch liczb całkowitych daje w obu systemach wynik całkowity (7/2 = 3). Mnożenie przez 100.0 lub 1.0 zamienia liczbę na dziesiętną i wynik ma ułamek.
- **CROSS JOIN LATERAL (VALUES ...) vs CROSS APPLY (VALUES ...)**: „Obraca” kolumny w wiersze (unpivot): z jednego wiersza z 7 kolumnami sygnałów robi 7 wierszy (nazwa sygnału, czy włączony). LATERAL pozwala małej tabeli VALUES korzystać z kolumn bieżącego wiersza. W SQL Server to samo robi CROSS APPLY.
- **(VALUES (30), (50), ...) AS th(threshold)**: Mała tabela wpisana ręcznie w zapytaniu, tu lista progów. AS th(threshold) nadaje jej nazwę i nazwę kolumny. Działa tak samo w obu systemach.
- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

03_risk_scoring.sql

1. Część 1: scoring v1 każdej transakcji (tabela fact_transaction_risk)

Co robi: Buduje nową tabelę z wynikiem ryzyka każdej transakcji. Kroki tx i features są takie same jak w 02_sql_analysis/04. Krok signals zapisuje 7 sygnałów jako wartości prawda/fałsz (dochodzi historia klienta z fact_customer_risk). Krok scored dodaje punkty za każdy włączony sygnał (wagi z planu: 20/15/20/15/10/30/10), a ostatni SELECT nadaje poziom LOW / MEDIUM / HIGH / CRITICAL. Na końcu tabela dostaje klucz główny.

Wynik na Twoich danych: 257 947 wierszy. Poziomy: LOW 220 315, MEDIUM 34 802, HIGH 2 143, CRITICAL 687 transakcji.

PostgreSQL (Twój skrypt)

```
DROP TABLE IF EXISTS fact_transaction_risk;

CREATE TABLE fact_transaction_risk AS
WITH tx AS (
    SELECT
        t.transaction_id, t.customer_id, t.tx_datetime, t.country, t.device_id, t.is_fraud,
        m.merchant_category,
        t.amount * fx.eur_rate                                  AS amount_eur
    FROM fact_transaction t
    JOIN dim_merchant m USING (merchant_id)
    JOIN dim_fx_rate fx ON fx.rate_date = t.tx_datetime::date AND fx.currency = t.currency
),
features AS (
    -- the same window functions as in 02_sql_analysis/04_window_functions.sql
    SELECT
        tx.*,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY tx_datetime, transaction_id)  AS tx_no,
        AVG(amount_eur) OVER (PARTITION BY customer_id ORDER BY tx_datetime, transaction_id
                              ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING)             AS hist_avg_eur,
        COUNT(*) OVER (PARTITION BY customer_id ORDER BY tx_datetime
                       RANGE BETWEEN INTERVAL '10 minutes' PRECEDING AND CURRENT ROW)       AS tx_last_10_min,
        MIN(tx_datetime) OVER (PARTITION BY customer_id, device_id)                         AS device_first_use,
        MIN(tx_datetime) OVER (PARTITION BY customer_id)                                    AS customer_first_tx,
        ROW_NUMBER() OVER (PARTITION BY customer_id, country ORDER BY tx_datetime, transaction_id) AS
tx_no_in_country
    FROM tx
),
signals AS (
    SELECT
        f.transaction_id, f.customer_id, f.tx_datetime, f.amount_eur, f.is_fraud,
        (f.amount_eur > 3 * f.hist_avg_eur)                                     AS s_amount_3x_avg,
        (f.device_first_use > f.customer_first_tx
         AND f.tx_datetime < f.device_first_use + INTERVAL '24 hours')          AS s_new_device,
        (f.tx_no > 1 AND f.tx_no_in_country = 1)                                AS s_new_country,
        (f.tx_last_10_min >= 5)                                                 AS s_5_tx_in_10_min,
        (f.merchant_category IN ('crypto', 'gift_cards', 'electronics'))        AS s_risky_category,
        (r.previous_fraud OR r.chargebacks > 0)                                 AS s_fraud_history,
        (EXTRACT(HOUR FROM f.tx_datetime) BETWEEN 0 AND 4)                      AS s_night_hour
    FROM features f
    LEFT JOIN fact_customer_risk r USING (customer_id)
),
scored AS (
    SELECT
        s.*,
        -- COALESCE: a signal that cannot be calculated (e.g. no history for the 1st transaction) = 0 points
          CASE WHEN COALESCE(s_amount_3x_avg,  false) THEN 20 ELSE 0 END
        + CASE WHEN s_new_device                      THEN 15 ELSE 0 END
        + CASE WHEN s_new_country                     THEN 20 ELSE 0 END
        + CASE WHEN s_5_tx_in_10_min                  THEN 15 ELSE 0 END
        + CASE WHEN s_risky_category                  THEN 10 ELSE 0 END
        + CASE WHEN COALESCE(s_fraud_history,  false) THEN 30 ELSE 0 END
        + CASE WHEN s_night_hour                      THEN 10 ELSE 0 END            AS risk_score
    FROM signals s
)
SELECT
    *,
    CASE WHEN risk_score >= 80 THEN 'CRITICAL'
         WHEN risk_score >= 60 THEN 'HIGH'
         WHEN risk_score >= 30 THEN 'MEDIUM'
         ELSE 'LOW' END                                                         AS risk_level
FROM scored;

ALTER TABLE fact_transaction_risk ADD PRIMARY KEY (transaction_id);
```

Microsoft SQL Server (SSMS)

```
DROP TABLE IF EXISTS fact_transaction_risk;
DROP TABLE IF EXISTS #tx;
```

```
-- Step 1: base data into a temp table with an index (for the 10-minute window)
SELECT
    t.transaction_id, t.customer_id, t.tx_datetime, t.country, t.device_id, t.is_fraud,
    m.merchant_category,
    t.amount * fx.eur_rate                                      AS amount_eur
INTO #tx
FROM fact_transaction t
JOIN dim_merchant m ON m.merchant_id = t.merchant_id
JOIN dim_fx_rate fx ON fx.rate_date = CAST(t.tx_datetime AS date) AND fx.currency = t.currency;

CREATE INDEX ix_tx_customer_time ON #tx (customer_id, tx_datetime);

-- Step 2: features, signals, score -> new table
WITH features AS (
    SELECT
        tx.*,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY tx_datetime, transaction_id)  AS tx_no,
        AVG(amount_eur) OVER (PARTITION BY customer_id ORDER BY tx_datetime, transaction_id
                              ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING)            AS hist_avg_eur,
        v10.tx_last_10_min,
        MIN(tx_datetime) OVER (PARTITION BY customer_id, device_id)                        AS device_first_use,
        MIN(tx_datetime) OVER (PARTITION BY customer_id)                                   AS customer_first_tx,
        ROW_NUMBER() OVER (PARTITION BY customer_id, country ORDER BY tx_datetime, transaction_id) AS
tx_no_in_country
    FROM #tx AS tx
    OUTER APPLY (SELECT COUNT(*) AS tx_last_10_min
                 FROM #tx t2
                 WHERE t2.customer_id = tx.customer_id
                   AND t2.tx_datetime BETWEEN DATEADD(minute, -10, tx.tx_datetime) AND tx.tx_datetime) v10
),
signals AS (
    SELECT
        f.transaction_id, f.customer_id, f.tx_datetime, f.amount_eur, f.is_fraud,
        -- T-SQL has no boolean expressions as values: CASE ... THEN 1 ELSE 0, stored as BIT
        CAST(CASE WHEN f.amount_eur > 3 * f.hist_avg_eur THEN 1 ELSE 0 END AS bit) AS s_amount_3x_avg,
        CAST(CASE WHEN f.device_first_use > f.customer_first_tx
                   AND f.tx_datetime < DATEADD(hour, 24, f.device_first_use)
              THEN 1 ELSE 0 END AS bit)                                            AS s_new_device,
        CAST(CASE WHEN f.tx_no > 1 AND f.tx_no_in_country = 1 THEN 1 ELSE 0 END AS bit) AS s_new_country,
        CAST(CASE WHEN f.tx_last_10_min >= 5 THEN 1 ELSE 0 END AS bit)               AS s_5_tx_in_10_min,
        CAST(CASE WHEN f.merchant_category IN ('crypto', 'gift_cards', 'electronics')
              THEN 1 ELSE 0 END AS bit)                                            AS s_risky_category,
        CAST(CASE WHEN r.previous_fraud = 1 OR r.chargebacks > 0 THEN 1 ELSE 0 END AS bit) AS s_fraud_history,
        CAST(CASE WHEN DATEPART(hour, f.tx_datetime) BETWEEN 0 AND 4 THEN 1 ELSE 0 END AS bit) AS s_night_hour
    FROM features f
    LEFT JOIN fact_customer_risk r ON r.customer_id = f.customer_id
),
scored AS (
    SELECT
        s.*,
          CASE WHEN s_amount_3x_avg = 1  THEN 20 ELSE 0 END
        + CASE WHEN s_new_device = 1     THEN 15 ELSE 0 END
        + CASE WHEN s_new_country = 1    THEN 20 ELSE 0 END
        + CASE WHEN s_5_tx_in_10_min = 1 THEN 15 ELSE 0 END
        + CASE WHEN s_risky_category = 1 THEN 10 ELSE 0 END
        + CASE WHEN s_fraud_history = 1  THEN 30 ELSE 0 END
        + CASE WHEN s_night_hour = 1     THEN 10 ELSE 0 END            AS risk_score
    FROM signals s
)
SELECT
    *,
    CASE WHEN risk_score >= 80 THEN 'CRITICAL'
         WHEN risk_score >= 60 THEN 'HIGH'
         WHEN risk_score >= 30 THEN 'MEDIUM'
         ELSE 'LOW' END                                                    AS risk_level
INTO fact_transaction_risk
FROM scored;

ALTER TABLE fact_transaction_risk ALTER COLUMN transaction_id BIGINT NOT NULL;
ALTER TABLE fact_transaction_risk
    ADD CONSTRAINT pk_fact_transaction_risk PRIMARY KEY (transaction_id);
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`CREATE TABLE x AS WITH ... SELECT ...`	`WITH ... SELECT ... INTO x FROM ...`	Utworzenie tabeli z wyniku zapytania. W T-SQL słowo INTO stoi w ostatnim SELECT-cie, przed FROM.
`RANGE BETWEEN INTERVAL '10 minutes'`	`OUTER APPLY + tabela #tx z indeksem`	Okno czasowe zastąpione podzapytaniem (patrz 02_sql_analysis/04, zapytanie 2).
`(a > b) AS s_... (BOOLEAN)`	`CAST(CASE WHEN a > b THEN 1 ELSE 0 END AS bit)`	W T-SQL warunek nie jest wartością, więc zamieniamy go na 1/0 i zapisujemy jako BIT.
`COALESCE(s_amount_3x_avg, false)`	`s_amount_3x_avg = 1`	W PostgreSQL porównanie z NULL-em (pierwsza transakcja klienta, brak średniej) daje NULL, dlatego COALESCE. W T-SQL CASE ... ELSE 0 od razu daje 0, więc COALESCE nie jest potrzebny.
`ADD PRIMARY KEY (transaction_id)`	`ALTER COLUMN ... NOT NULL + ADD CONSTRAINT nazwa PRIMARY KEY`	SQL Server wymaga, żeby kolumna klucza była NOT NULL; SELECT INTO nie zawsze to przenosi.
`USING / ::date / EXTRACT / INTERVAL`	`ON / CAST / DATEPART / DATEADD`	Jak w poprzednich zapytaniach.

Funkcje i metody użyte w tym zapytaniu

- **DROP TABLE IF EXISTS**: Usuwa tabelę, jeśli istnieje; jeśli nie, nie zgłasza błędu. Dzięki temu skrypt można uruchamiać wiele razy. W SQL Server działa od wersji 2016.
- **CREATE TABLE ... AS SELECT vs SELECT ... INTO**: Tworzy nową tabelę od razu wypełnioną wynikiem zapytania; kolumny i typy biorą się z SELECT-a. W PostgreSQL piszemy CREATE TABLE nazwa AS SELECT ..., a w SQL Server SELECT ... INTO nazwa FROM ... (słowo INTO stoi po liście kolumn, przed FROM).
- **Tabela tymczasowa #nazwa (T-SQL)**: Tabela, która istnieje tylko do końca sesji w SSMS. Zapisujemy do niej wynik pośredni i zakładamy indeks, żeby kolejne kroki (OUTER APPLY) działały szybko. W PostgreSQL odpowiednikiem jest CREATE TEMP TABLE, ale tam nie było potrzebne, bo okno RANGE robi to samo w jednym przebiegu.
- **CREATE INDEX**: Indeks to „spis treści” dla kolumn: przyspiesza wyszukiwanie wierszy (tu: transakcji danego klienta w danym czasie).
- **WITH nazwa AS (...) (CTE)**: „Tymczasowa tabela na czas jednego zapytania”. Dzielimy duże zapytanie na nazwane kroki, a każdy kolejny krok może korzystać z poprzednich. Działa tak samo w obu systemach.
- **JOIN ... ON**: Łączy wiersze z dwóch tabel według warunku, np. transakcja z kursem waluty z tego samego dnia. Zostają tylko wiersze, które mają parę (INNER JOIN).
- **LEFT JOIN**: Jak JOIN, ale zostawia wszystkie wiersze z lewej tabeli; tam, gdzie nie ma pary, kolumny z prawej tabeli są puste (NULL). Używamy np. do znalezienia transakcji bez kursu waluty.
- **JOIN ... USING (kolumna)**: Skrót w PostgreSQL: gdy obie tabele mają kolumnę o tej samej nazwie, USING (customer_id) znaczy ON a.customer_id = b.customer_id. SQL Server nie zna USING, więc trzeba pisać pełne ON.
- **::date vs CAST(... AS date)**: Zamiana typu. W PostgreSQL skrót x::date (tu: obcina godzinę z daty-czasu). W SQL Server CAST(x AS date) lub CONVERT(date, x). Zapis CAST działa też w PostgreSQL.
- **Funkcja okna: ... OVER (PARTITION BY ... ORDER BY ...)**: Liczy wartość dla każdego wiersza na podstawie „okna” innych wierszy, ale nie skleja wierszy w grupy (w przeciwieństwie do GROUP BY). PARTITION BY dzieli dane na osobne okna (np. osobno dla każdego klienta), ORDER BY ustala kolejność w oknie. Składnia jest taka sama w obu systemach.
- **ROW_NUMBER()**: Numeruje wiersze w oknie 1, 2, 3... według ORDER BY. Tu: która to transakcja klienta (1 = pierwsza) albo pierwsza transakcja w danym kraju.
- **ROWS BETWEEN ... PRECEDING AND ...**: Ramka okna liczona w wierszach. UNBOUNDED PRECEDING AND 1 PRECEDING = wszystkie wcześniejsze wiersze bez bieżącego (średnia „przed” transakcją, bez podglądania wyniku). 6 PRECEDING AND CURRENT ROW = 7 ostatnich dni (średnia krocząca). Tak samo w obu systemach.
- **RANGE BETWEEN INTERVAL '10 minutes' PRECEDING AND CURRENT ROW**: Ramka okna liczona w CZASIE, nie w wierszach: wszystkie transakcje klienta z ostatnich 10 minut. To jedna z największych przewag PostgreSQL. SQL Server tego nie obsługuje, więc liczymy to podzapytaniem w OUTER APPLY z warunkiem na czas (BETWEEN DATEADD(minute, -10, ts) AND ts).
- **OUTER APPLY (podzapytanie)**: T-SQL: dla każdego wiersza uruchamia podzapytanie, które może korzystać z jego kolumn (tu: policz transakcje tego klienta z ostatnich 10 minut). OUTER zostawia wiersz, nawet gdy podzapytanie nic nie zwróci. Odpowiednik w PostgreSQL: LEFT JOIN LATERAL.
- **MIN(x) OVER (PARTITION BY ...)**: Najmniejsza wartość w całym oknie, dopisana do każdego wiersza. Tu: kiedy klient pierwszy raz użył danego urządzenia i kiedy zrobił pierwszą transakcję w ogóle.
- **Kolumna z warunku: (a > b) AS sygnał**: W PostgreSQL wynik porównania jest wartością BOOLEAN i może być kolumną. W SQL Server porównanie nie jest wartością: trzeba napisać CASE WHEN a > b THEN 1 ELSE 0 END (i ewentualnie CAST(... AS bit)).

- **BOOLEAN vs BIT**: PostgreSQL ma prawdziwy typ logiczny BOOLEAN z wartościami true / false. SQL Server go nie ma: używa BIT z wartościami 1 / 0. Dlatego w T-SQL zawsze piszemy porównanie, np. is_fraud = 1, a w PostgreSQL wystarczy samo is_fraud.
- **INTERVAL '24 hours' vs DATEADD**: PostgreSQL ma typ „odcinek czasu”: ts + INTERVAL '24 hours' to data przesunięta o dobę. SQL Server nie ma takiego typu; używamy DATEADD(hour, 24, ts) (albo minute, day).
- **EXTRACT(HOUR FROM ts) vs DATEPART(hour, ts)**: Wyciąga część daty, tu godzinę 0–23.
- **IN ('a', 'b', ...)**: Czy wartość jest jedną z listy; krótszy zapis wielu OR.
- **AND / OR / NOT**: Łączenie warunków: oba naraz (AND), choć jeden (OR), zaprzeczenie (NOT). W PostgreSQL NOT is_fraud działa na kolumnie BOOLEAN; w T-SQL piszemy is_fraud = 0.
- **COALESCE(x, y) / ISNULL(x, y)**: Zwraca pierwszą niepustą wartość, np. 0 zamiast NULL. COALESCE działa w obu systemach; ISNULL to wersja tylko z SQL Server (dokładnie 2 argumenty).
- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli”: dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **ALTER TABLE ... ADD PRIMARY KEY**: Dodaje klucz główny do istniejącej tabeli. W SQL Server kolumna musi być najpierw oznaczona jako NOT NULL, a klucz zwykle dostaje nazwę (ADD CONSTRAINT nazwa PRIMARY KEY).

2. Część 2: wynik ryzyka klienta (UPDATE fact_customer_risk)

Co robi: Wynik klienta = najwyższy wynik jego transakcji z ostatnich 30 dni okresu (od 1.06.2026). Podzapytanie c liczy to maksimum dla każdego klienta; LEFT JOIN sprawia, że klienci bez transakcji w tym czasie też są na liście (z NULL, który COALESCE zamienia na 0). UPDATE wpisuje wynik i poziom do fact_customer_risk.

Wynik na Twoich danych: 10 000 klientów: LOW 7 734, MEDIUM 2 066, HIGH 143, CRITICAL 57 (v1).

PostgreSQL (Twój skrypt)

```
UPDATE fact_customer_risk r
SET risk_score = COALESCE(c.max_score, 0),
    risk_level = CASE WHEN COALESCE(c.max_score, 0) >= 80 THEN 'CRITICAL'
                      WHEN COALESCE(c.max_score, 0) >= 60 THEN 'HIGH'
                      WHEN COALESCE(c.max_score, 0) >= 30 THEN 'MEDIUM'
                      ELSE 'LOW' END
FROM (
    SELECT cu.customer_id, MAX(tr.risk_score) AS max_score
    FROM dim_customer cu
    LEFT JOIN fact_transaction_risk tr
           ON tr.customer_id = cu.customer_id
          AND tr.tx_datetime >= TIMESTAMP '2026-07-01' - INTERVAL '30 days'
    GROUP BY cu.customer_id
) c
WHERE c.customer_id = r.customer_id;
```

Microsoft SQL Server (SSMS)

```
UPDATE r
SET risk_score = COALESCE(c.max_score, 0),
    risk_level = CASE WHEN COALESCE(c.max_score, 0) >= 80 THEN 'CRITICAL'
                      WHEN COALESCE(c.max_score, 0) >= 60 THEN 'HIGH'
                      WHEN COALESCE(c.max_score, 0) >= 30 THEN 'MEDIUM'
                      ELSE 'LOW' END
FROM fact_customer_risk r
JOIN (
    SELECT cu.customer_id, MAX(tr.risk_score) AS max_score
    FROM dim_customer cu
    LEFT JOIN fact_transaction_risk tr
           ON tr.customer_id = cu.customer_id
          AND tr.tx_datetime >= DATEADD(day, -30, CAST('2026-07-01' AS datetime2))
    GROUP BY cu.customer_id
) c ON c.customer_id = r.customer_id;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`UPDATE tabela r SET ... FROM (...) c WHERE c.id = r.id`	`UPDATE r SET ... FROM tabela r JOIN (...) c ON c.id = r.id`	W T-SQL po UPDATE stoi alias, a tabela docelowa i podzapytanie są połączone zwykłym JOIN w części FROM.
`TIMESTAMP '2026-07-01' - INTERVAL '30 days'`	`DATEADD(day, -30, CAST('2026-07-01' AS datetime2))`	Data 30 dni przed końcem okresu (1.06.2026).

Funkcje i metody użyte w tym zapytaniu

- **UPDATE ... SET**: Zmienia wartości w istniejących wierszach. Bez WHERE zmienia wszystkie wiersze tabeli.

- **UPDATE ... FROM (aktualizacja z innej tabeli)**: Aktualizuje tabelę wartościami policzonymi w innym zapytaniu. PostgreSQL: UPDATE tabela alias SET ... FROM (podzapytanie) c WHERE c.id = alias.id. SQL Server: UPDATE alias SET ... FROM tabela alias JOIN (podzapytanie) c ON c.id = alias.id, czyli warunek łączenia jest w JOIN, a po UPDATE stoi alias.
- **COALESCE(x, y) / ISNULL(x, y)**: Zwraca pierwszą niepustą wartość, np. 0 zamiast NULL. COALESCE działa w obu systemach; ISNULL to wersja tylko z SQL Server (dokładnie 2 argumenty).
- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli": dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **MIN(x) / MAX(x)**: Najmniejsza / największa wartość w grupie (np. najwyższy wynik ryzyka klienta).
- **LEFT JOIN**: Jak JOIN, ale zostawia wszystkie wiersze z lewej tabeli; tam, gdzie nie ma pary, kolumny z prawej tabeli są puste (NULL). Używamy np. do znalezienia transakcji bez kursu waluty.
- **Stała data: DATE '2026-01-01'**: W PostgreSQL DATE '...' i TIMESTAMP '...' tworzą wartość daty z tekstu. W SQL Server wystarczy tekst '2026-01-01' (porównywany z kolumną daty) albo CAST('2026-01-01' AS date).
- **INTERVAL '24 hours' vs DATEADD**: PostgreSQL ma typ „odcinek czasu": ts + INTERVAL '24 hours' to data przesunięta o dobę. SQL Server nie ma takiego typu; używamy DATEADD(hour, 24, ts) (albo minute, day).
- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.

3. Kontrola 3a: transakcje według poziomu ryzyka

Co robi: Ile transakcji jest na każdym poziomie, ile z nich to naprawdę fraud, jaki to procent i jaką część WSZYSTKICH fraudów zawiera poziom. Ostatnia kolumna używa funkcji okna na wyniku grupowania: SUM(...) OVER () to suma fraudów ze wszystkich poziomów. Sortowanie po MIN(risk_score) ustawia poziomy w kolejności LOW → CRITICAL.

Wynik na Twoich danych: Fraud rate: LOW 0,26%, MEDIUM 5,32%, HIGH 56%, CRITICAL 90,39%.

PostgreSQL (Twój skrypt)

```
SELECT
    risk_level,
    COUNT(*)                                          AS transactions,
    COUNT(*) FILTER (WHERE is_fraud)                  AS fraud_transactions,
    ROUND(100.0 * AVG(is_fraud::int), 2)              AS fraud_rate_pct,
    ROUND(100.0 * COUNT(*) FILTER (WHERE is_fraud)
                / SUM(COUNT(*) FILTER (WHERE is_fraud)) OVER (), 1) AS share_of_all_fraud_pct
FROM fact_transaction_risk
GROUP BY risk_level
ORDER BY MIN(risk_score);
```

Microsoft SQL Server (SSMS)

```
SELECT
    risk_level,
    COUNT(*)                                          AS transactions,
    SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END)     AS fraud_transactions,
    ROUND(100.0 * AVG(CAST(is_fraud AS float)), 2)    AS fraud_rate_pct,
    ROUND(100.0 * SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END)
                / SUM(SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END)) OVER (), 1) AS share_of_all_fraud_pct
FROM fact_transaction_risk
GROUP BY risk_level
ORDER BY MIN(risk_score);
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`COUNT(*) FILTER / AVG(::int)`	`SUM(CASE ...) / AVG(CAST AS float)`	Liczenie fraudów. SUM(SUM(...)) OVER () też działa w T-SQL: okno liczone na zgrupowanym wyniku.

Funkcje i metody użyte w tym zapytaniu

- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AVG(is_fraud::int) vs AVG(CAST(is_fraud AS float))**: Średnia z 0/1 to udział jedynek, czyli fraud rate. W PostgreSQL zamieniamy BOOLEAN na liczbę (::int). W SQL Server BIT trzeba zamienić na float, bo AVG z liczby całkowitej dałoby wynik całkowity (0).
- **100.0 * a / b (dzielenie z ułamkiem)**: Dzielenie dwóch liczb całkowitych daje w obu systemach wynik całkowity (7/2 = 3). Mnożenie przez 100.0 lub 1.0 zamienia liczbę na dziesiętną i wynik ma ułamek.

- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **SUM(...) OVER () na wyniku GROUP BY**: Funkcja okna liczona na już zgrupowanych wierszach: OVER () = cała tabela wyników. Tu: suma fraudów ze wszystkich poziomów, żeby policzyć udział każdego poziomu. Działa w obu systemach.
- **MIN(x) / MAX(x)**: Najmniejsza / największa wartość w grupie (np. najwyższy wynik ryzyka klienta).
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

4. Kontrola 3b: klienci według poziomu ryzyka

Co robi: Ilu klientów jest na każdym poziomie (stan na 30.06.2026) i jaki mają średni wynik.

Wynik na Twoich danych: LOW 7 734, MEDIUM 2 066, HIGH 143, CRITICAL 57 klientów.

PostgreSQL (Twój skrypt)

```
SELECT
    risk_level,
    COUNT(*)                                        AS customers,
    ROUND(AVG(risk_score), 1)                       AS avg_score
FROM fact_customer_risk
GROUP BY risk_level
ORDER BY MIN(risk_score);
```

Microsoft SQL Server (SSMS)

```
SELECT
    risk_level,
    COUNT(*)                                        AS customers,
    ROUND(AVG(risk_score), 1)                       AS avg_score
FROM fact_customer_risk
GROUP BY risk_level
ORDER BY MIN(risk_score);
```

Różnice

PostgreSQL	SQL Server	Dlaczego
–	–	Zapytanie jest identyczne w obu systemach. risk_score ma typ NUMERIC(5,2), więc AVG nie obcina ułamka także w T-SQL.

Funkcje i metody użyte w tym zapytaniu

- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **AVG(x)**: Średnia arytmetyczna. Uwaga w SQL Server: AVG z kolumny całkowitej (INT, wynik COUNT) zwraca liczbę całkowitą z obciętym ułamkiem (np. 25 zamiast 25,8). Dlatego w T-SQL mnożymy przez 1.0 albo rzutujemy na float.
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **MIN(x) / MAX(x)**: Najmniejsza / największa wartość w grupie (np. najwyższy wynik ryzyka klienta).
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

04_false_positives.sql

1. Macierz pomyłek dla progu 60

Co robi: Dzieli transakcje na 4 grupy: wiersze to decyzja systemu (alarm przy wyniku ≥ 60 albo brak), kolumny to rzeczywistość (fraud albo OK). Przekątna to trafne decyzje (TP, TN), poza nią pomyłki (FP = fałszywy alarm, FN = przepuszczony fraud).

Wynik na Twoich danych: System: fraud → 1 821 fraudów (TP), 1 009 OK (FP). System: OK → 2 421 fraudów (FN), 252 696 OK (TN).

PostgreSQL (Twój skrypt)

```
SELECT
    CASE WHEN risk_score >= 60 THEN 'System: fraud' ELSE 'System: OK' END   AS system_decision,
    COUNT(*) FILTER (WHERE is_fraud)                                        AS reality_fraud,
    COUNT(*) FILTER (WHERE NOT is_fraud)                                    AS reality_ok
FROM fact_transaction_risk
GROUP BY 1
ORDER BY 1 DESC;
```

Microsoft SQL Server (SSMS)

```
SELECT
    CASE WHEN risk_score >= 60 THEN 'System: fraud' ELSE 'System: OK' END   AS system_decision,
    SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END)                           AS reality_fraud,
    SUM(CASE WHEN is_fraud = 0 THEN 1 ELSE 0 END)                           AS reality_ok
FROM fact_transaction_risk
GROUP BY CASE WHEN risk_score >= 60 THEN 'System: fraud' ELSE 'System: OK' END
ORDER BY 1 DESC;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`FILTER (WHERE NOT is_fraud)`	`SUM(CASE WHEN is_fraud = 0 ...)`	NOT na kolumnie BOOLEAN zastępujemy porównaniem z 0.
`GROUP BY 1`	`GROUP BY CASE ... END`	Brak grupowania po numerze kolumny.

Funkcje i metody użyte w tym zapytaniu

- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli": dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AND / OR / NOT**: Łączenie warunków: oba naraz (AND), choć jeden (OR), zaprzeczenie (NOT). W PostgreSQL NOT is_fraud działa na kolumnie BOOLEAN; w T-SQL piszemy is_fraud = 0.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **GROUP BY 1 (numer kolumny)**: W PostgreSQL można grupować po numerze kolumny z SELECT-a. SQL Server na to nie pozwala: trzeba powtórzyć całe wyrażenie (np. cały CASE) albo policzyć je wcześniej w podzapytaniu / CROSS APPLY i grupować po aliasie.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

2. Tabela progów 30 / 50 / 60 / 70 / 80

Co robi: Ta sama macierz pomyłek, ale policzona dla 5 progów naraz. Lista progów to mała tabela VALUES; CROSS JOIN sprawdza każdą transakcję przy każdym progu. Krok matrix liczy TP, FP, FN, TN dla każdego progu, a ostatni SELECT liczy z nich wskaźniki: precision (trafność alarmów), recall (wykrywalność), false positive rate i liczbę fałszywych alarmów na jeden złapany fraud. NULLIF chroni przed dzieleniem przez zero.

Wynik na Twoich danych: Próg 60: precision 64,3%, recall 42,9%, FPR 0,40%. Im wyższy próg, tym wyższa precision i niższy recall.

PostgreSQL (Twój skrypt)

```
WITH thresholds AS (
    SELECT threshold FROM (VALUES (30), (50), (60), (70), (80)) AS th(threshold)
),
matrix AS (
    SELECT
        th.threshold,
        COUNT(*) FILTER (WHERE r.risk_score >= th.threshold)                      AS flagged,
        COUNT(*) FILTER (WHERE r.risk_score >= th.threshold AND r.is_fraud)       AS tp,
        COUNT(*) FILTER (WHERE r.risk_score >= th.threshold AND NOT r.is_fraud)   AS fp,
        COUNT(*) FILTER (WHERE r.risk_score <  th.threshold AND r.is_fraud)       AS fn,
        COUNT(*) FILTER (WHERE r.risk_score <  th.threshold AND NOT r.is_fraud)   AS tn
    FROM thresholds th
    CROSS JOIN fact_transaction_risk r                  -- every transaction checked against every threshold
    GROUP BY th.threshold
)
SELECT
    threshold,
    flagged,
    tp AS fraud_detected,
    fp AS false_alarms,
    fn AS fraud_missed,
    ROUND(100.0 * tp / NULLIF(tp + fp, 0), 1)           AS precision_pct,
    ROUND(100.0 * tp / NULLIF(tp + fn, 0), 1)           AS recall_pct,
    ROUND(100.0 * fp / NULLIF(fp + tn, 0), 2)           AS false_positive_rate_pct,
    ROUND(1.0 * fp / NULLIF(tp, 0), 1)                  AS false_alarms_per_fraud
FROM matrix
ORDER BY threshold;
```

Microsoft SQL Server (SSMS)

```
WITH thresholds AS (
    SELECT threshold FROM (VALUES (30), (50), (60), (70), (80)) AS th(threshold)
),
matrix AS (
    SELECT
        th.threshold,
        SUM(CASE WHEN r.risk_score >= th.threshold THEN 1 ELSE 0 END)                    AS flagged,
        SUM(CASE WHEN r.risk_score >= th.threshold AND r.is_fraud = 1 THEN 1 ELSE 0 END) AS tp,
        SUM(CASE WHEN r.risk_score >= th.threshold AND r.is_fraud = 0 THEN 1 ELSE 0 END) AS fp,
        SUM(CASE WHEN r.risk_score <  th.threshold AND r.is_fraud = 1 THEN 1 ELSE 0 END) AS fn,
        SUM(CASE WHEN r.risk_score <  th.threshold AND r.is_fraud = 0 THEN 1 ELSE 0 END) AS tn
    FROM thresholds th
    CROSS JOIN fact_transaction_risk r
    GROUP BY th.threshold
)
SELECT
    threshold,
    flagged,
    tp AS fraud_detected,
    fp AS false_alarms,
    fn AS fraud_missed,
    ROUND(100.0 * tp / NULLIF(tp + fp, 0), 1)           AS precision_pct,
    ROUND(100.0 * tp / NULLIF(tp + fn, 0), 1)           AS recall_pct,
    ROUND(100.0 * fp / NULLIF(fp + tn, 0), 2)           AS false_positive_rate_pct,
    ROUND(1.0 * fp / NULLIF(tp, 0), 1)                  AS false_alarms_per_fraud
FROM matrix
ORDER BY threshold;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`COUNT(*) FILTER (WHERE ... AND r.is_fraud)`	`SUM(CASE WHEN ... AND r.is_fraud = 1 THEN 1 ELSE 0 END)`	Jedyna różnica: liczenie warunkowe. VALUES, CROSS JOIN, NULLIF i 100.0 działają identycznie.

Funkcje i metody użyte w tym zapytaniu

- **WITH nazwa AS (...) (CTE)**: „Tymczasowa tabela na czas jednego zapytania". Dzielimy duże zapytanie na nazwane kroki, a każdy kolejny krok może korzystać z poprzednich. Działa tak samo w obu systemach.

- **(VALUES (30), (50), ...) AS th(threshold)**: Mała tabela wpisana ręcznie w zapytaniu, tu lista progów. AS th(threshold) nadaje jej nazwę i nazwę kolumny. Działa tak samo w obu systemach.
- **CROSS JOIN**: Łączy każdy wiersz z każdym (iloczyn kartezjański): 5 progów × 257 947 transakcji = każda transakcja sprawdzona przy każdym progu.
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AND / OR / NOT**: Łączenie warunków: oba naraz (AND), choć jeden (OR), zaprzeczenie (NOT). W PostgreSQL NOT is_fraud działa na kolumnie BOOLEAN; w T-SQL piszemy is_fraud = 0.
- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **100.0 * a / b (dzielenie z ułamkiem)**: Dzielenie dwóch liczb całkowitych daje w obu systemach wynik całkowity (7/2 = 3). Mnożenie przez 100.0 lub 1.0 zamienia liczbę na dziesiętną i wynik ma ułamek.
- **NULLIF(x, 0)**: Zwraca NULL, gdy x = 0. Dzielenie przez NULL daje NULL zamiast błędu „dzielenie przez zero”. Tak samo w obu systemach.
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

3. Które reguły robią fałszywe alarmy? (próg 60)

Co robi: Bierze tylko alarmy (wynik ≥ 60). CROSS JOIN LATERAL zamienia 7 kolumn sygnałów na 7 wierszy (unpivot). Dla każdego sygnału liczymy, w ilu fałszywych alarmach był włączony i jaki to procent wszystkich fałszywych alarmów, oraz to samo dla trafionych alarmów. Sygnał częstszy w fałszywych niż w trafionych to kandydat do obniżenia wagi.

Wynik na Twoich danych: Historia fraudu/chargebacku: 95,8% fałszywych alarmów vs 61,1% trafionych (do obniżenia). Nowe urządzenie: 10,3% vs 86,2% (do podniesienia).

PostgreSQL (Twój skrypt)

```
WITH alerts AS (
    SELECT *
    FROM fact_transaction_risk
    WHERE risk_score >= 60
)
SELECT
    v.signal,
    COUNT(*) FILTER (WHERE v.is_on AND NOT a.is_fraud)                          AS in_false_alarms,
    ROUND(100.0 * COUNT(*) FILTER (WHERE v.is_on AND NOT a.is_fraud)
                / NULLIF(COUNT(*) FILTER (WHERE NOT a.is_fraud), 0), 1)        AS pct_of_false_alarms,
    COUNT(*) FILTER (WHERE v.is_on AND a.is_fraud)                              AS in_true_alerts,
    ROUND(100.0 * COUNT(*) FILTER (WHERE v.is_on AND a.is_fraud)
                / NULLIF(COUNT(*) FILTER (WHERE a.is_fraud), 0), 1)            AS pct_of_true_alerts
FROM alerts a
CROSS JOIN LATERAL (VALUES                      -- signal columns -> rows (T-SQL: CROSS APPLY (VALUES ...))
    ('amount > 3x customer avg', COALESCE(a.s_amount_3x_avg, false)),
    ('new device',               a.s_new_device),
    ('new country',              a.s_new_country),
    ('5+ tx in 10 min',          a.s_5_tx_in_10_min),
    ('risky category',           a.s_risky_category),
    ('fraud / chargeback history', COALESCE(a.s_fraud_history, false)),
    ('night hour (0-4)',         a.s_night_hour)
) AS v(signal, is_on)
GROUP BY v.signal
ORDER BY in_false_alarms DESC;
```

Microsoft SQL Server (SSMS)

```
WITH alerts AS (
    SELECT *
    FROM fact_transaction_risk
    WHERE risk_score >= 60
)
SELECT
    v.signal,
    SUM(CASE WHEN v.is_on = 1 AND a.is_fraud = 0 THEN 1 ELSE 0 END)             AS in_false_alarms,
    ROUND(100.0 * SUM(CASE WHEN v.is_on = 1 AND a.is_fraud = 0 THEN 1 ELSE 0 END)
                / NULLIF(SUM(CASE WHEN a.is_fraud = 0 THEN 1 ELSE 0 END), 0), 1) AS pct_of_false_alarms,
    SUM(CASE WHEN v.is_on = 1 AND a.is_fraud = 1 THEN 1 ELSE 0 END)             AS in_true_alerts,
    ROUND(100.0 * SUM(CASE WHEN v.is_on = 1 AND a.is_fraud = 1 THEN 1 ELSE 0 END)
                / NULLIF(SUM(CASE WHEN a.is_fraud = 1 THEN 1 ELSE 0 END), 0), 1) AS pct_of_true_alerts
FROM alerts a
```

```
CROSS APPLY (VALUES
    ('amount > 3x customer avg', ISNULL(a.s_amount_3x_avg, 0)),
    ('new device',               a.s_new_device),
    ('new country',              a.s_new_country),
    ('5+ tx in 10 min',          a.s_5_tx_in_10_min),
    ('risky category',           a.s_risky_category),
    ('fraud / chargeback history', ISNULL(a.s_fraud_history, 0)),
    ('night hour (0-4)',         a.s_night_hour)
) AS v(signal, is_on)
GROUP BY v.signal
ORDER BY in_false_alarms DESC;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`CROSS JOIN LATERAL (VALUES ...)`	`CROSS APPLY (VALUES ...)`	Unpivot: kolumny sygnałów → wiersze.
`v.is_on AND NOT a.is_fraud`	`v.is_on = 1 AND a.is_fraud = 0`	Warunki na BIT zawsze z porównaniem.
`COALESCE(x, false)`	`ISNULL(x, 0)`	Pusty sygnał liczymy jako wyłączony. W wersji T-SQL tabeli z 03 sygnały nie mają NULL-i, więc ISNULL jest tylko zabezpieczeniem.

Funkcje i metody użyte w tym zapytaniu

- **WITH nazwa AS (...) (CTE)**: „Tymczasowa tabela na czas jednego zapytania”. Dzielimy duże zapytanie na nazwane kroki, a każdy kolejny krok może korzystać z poprzednich. Działa tak samo w obu systemach.
- **SELECT * / alias.***: Wszystkie kolumny tabeli lub kroku CTE.
- **CROSS JOIN LATERAL (VALUES ...) vs CROSS APPLY (VALUES ...)**: „Obraca” kolumny w wiersze (unpivot): z jednego wiersza z 7 kolumnami sygnałów robi 7 wierszy (nazwa sygnału, czy włączony). LATERAL pozwala małej tabeli VALUES korzystać z kolumn bieżącego wiersza. W SQL Server to samo robi CROSS APPLY.
- **(VALUES (30), (50), ...) AS th(threshold)**: Mała tabela wpisana ręcznie w zapytaniu, tu lista progów. AS th(threshold) nadaje jej nazwę i nazwę kolumny. Działa tak samo w obu systemach.
- **COALESCE(x, y) / ISNULL(x, y)**: Zwraca pierwszą niepustą wartość, np. 0 zamiast NULL. COALESCE działa w obu systemach; ISNULL to wersja tylko z SQL Server (dokładnie 2 argumenty).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AND / OR / NOT**: Łączenie warunków: oba naraz (AND), choć jeden (OR), zaprzeczenie (NOT). W PostgreSQL NOT is_fraud działa na kolumnie BOOLEAN; w T-SQL piszemy is_fraud = 0.
- **100.0 * a / b (dzielenie z ułamkiem)**: Dzielenie dwóch liczb całkowitych daje w obu systemach wynik całkowity (7/2 = 3). Mnożenie przez 100.0 lub 1.0 zamienia liczbę na dziesiętną i wynik ma ułamek.
- **NULLIF(x, 0)**: Zwraca NULL, gdy x = 0. Dzielenie przez NULL daje NULL zamiast błędu „dzielenie przez zero”. Tak samo w obu systemach.
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

4. Najczęstsze kombinacje sygnałów w alarmach (próg 60)

Co robi: Dla każdego alarmu skleja nazwy włączonych sygnałów w jeden napis (np. „amount 3x + risky category + history”). CASE bez ELSE zwraca NULL dla wyłączonego sygnału, a CONCAT_WS pomija NULL-e. Grupujemy po tym napisie i pokazujemy 10 kombinacji z największą liczbą fałszywych alarmów.

Wynik na Twoich danych: Najwięcej fałszywych alarmów: amount 3x + risky category + history (445 z 479). Wzorzec przejęcia konta amount 3x + new device + new country + risky category: 91,5% fraudu.

PostgreSQL (Twój skrypt)

```
SELECT
    CONCAT_WS(' + ',                              -- skips NULLs, the same in T-SQL (SQL Server 2017+)
        CASE WHEN s_amount_3x_avg   THEN 'amount 3x' END,
        CASE WHEN s_new_device      THEN 'new device' END,
        CASE WHEN s_new_country     THEN 'new country' END,
        CASE WHEN s_5_tx_in_10_min  THEN '5+ tx/10min' END,
        CASE WHEN s_risky_category  THEN 'risky category' END,
        CASE WHEN s_fraud_history   THEN 'history' END,
        CASE WHEN s_night_hour      THEN 'night' END)                 AS signals,
    COUNT(*)                                                          AS alerts,
    COUNT(*) FILTER (WHERE NOT is_fraud)                              AS false_alarms,
    ROUND(100.0 * AVG(is_fraud::int), 1)                              AS fraud_rate_pct
FROM fact_transaction_risk
WHERE risk_score >= 60
GROUP BY 1
ORDER BY false_alarms DESC
LIMIT 10;
```

Microsoft SQL Server (SSMS)

```
SELECT TOP 10
    x.signals,
    COUNT(*)                                                          AS alerts,
    SUM(CASE WHEN r.is_fraud = 0 THEN 1 ELSE 0 END)                   AS false_alarms,
    ROUND(100.0 * AVG(CAST(r.is_fraud AS float)), 1)                  AS fraud_rate_pct
FROM fact_transaction_risk r
CROSS APPLY (SELECT
    CONCAT_WS(' + ',                              -- SQL Server 2017+
        CASE WHEN r.s_amount_3x_avg = 1   THEN 'amount 3x' END,
        CASE WHEN r.s_new_device = 1      THEN 'new device' END,
        CASE WHEN r.s_new_country = 1     THEN 'new country' END,
        CASE WHEN r.s_5_tx_in_10_min = 1  THEN '5+ tx/10min' END,
        CASE WHEN r.s_risky_category = 1  THEN 'risky category' END,
        CASE WHEN r.s_fraud_history = 1   THEN 'history' END,
        CASE WHEN r.s_night_hour = 1      THEN 'night' END) AS signals) x
WHERE r.risk_score >= 60
GROUP BY x.signals
ORDER BY false_alarms DESC;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`GROUP BY 1 (po napisie z CONCAT_WS)`	`CROSS APPLY (SELECT CONCAT_WS(...) AS signals) x + GROUP BY x.signals`	Napis liczymy raz w CROSS APPLY i grupujemy po jego nazwie, zamiast powtarzać całe wyrażenie w GROUP BY.
`CASE WHEN s_new_device THEN`	`CASE WHEN r.s_new_device = 1 THEN`	Warunek na BIT.
`LIMIT 10`	`SELECT TOP 10`	Ograniczenie liczby wierszy.
`FILTER / AVG(::int)`	`SUM(CASE ...) / AVG(CAST AS float)`	Liczenie fraudów.

Funkcje i metody użyte w tym zapytaniu

- **CONCAT_WS(' + ', a, b, ...)**: Skleja teksty z separatorem i pomija puste (NULL). Tu z nazw włączonych sygnałów robi napis typu „amount 3x + risky category + history”. W SQL Server od wersji 2017.
- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli”: dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AVG(is_fraud::int) vs AVG(CAST(is_fraud AS float))**: Średnia z 0/1 to udział jedynek, czyli fraud rate. W PostgreSQL zamieniamy BOOLEAN na liczbę (::int). W SQL Server BIT trzeba zamienić na float, bo AVG z liczby całkowitej dałoby wynik całkowity (0).

- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **GROUP BY 1 (numer kolumny)**: W PostgreSQL można grupować po numerze kolumny z SELECT-a. SQL Server na to nie pozwala: trzeba powtórzyć całe wyrażenie (np. cały CASE) albo policzyć je wcześniej w podzapytaniu / CROSS APPLY i grupować po aliasie.
- **OUTER APPLY (podzapytanie)**: T-SQL: dla każdego wiersza uruchamia podzapytanie, które może korzystać z jego kolumn (tu: policz transakcje tego klienta z ostatnich 10 minut). OUTER zostawia wiersz, nawet gdy podzapytanie nic nie zwróci. Odpowiednik w PostgreSQL: LEFT JOIN LATERAL.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).
- **LIMIT n vs TOP n**: Zwraca tylko pierwsze n wierszy (po sortowaniu). PostgreSQL: LIMIT n na końcu zapytania. SQL Server: SELECT TOP n tuż po SELECT.

05_risk_score_v2.sql

1. Część 1: wynik v2 każdej transakcji (nowe kolumny)

Co robi: Dodaje do fact_transaction_risk dwie kolumny (risk_score_v2, risk_level_v2), o ile jeszcze ich nie ma, i wypełnia je. Wynik v2 liczymy z tych samych sygnałów co v1, tylko z nowymi wagami: nowe urządzenie 25 zamiast 15, historia 15 zamiast 30. Poziom nadaje drugi UPDATE (progi bez zmian).

Wynik na Twoich danych: Kolumny v2 obok v1 w tej samej tabeli; v1 zostaje do porównania.

PostgreSQL (Twój skrypt)

```
ALTER TABLE fact_transaction_risk
    ADD COLUMN IF NOT EXISTS risk_score_v2 INT,
    ADD COLUMN IF NOT EXISTS risk_level_v2 TEXT;

UPDATE fact_transaction_risk
SET risk_score_v2 =
      CASE WHEN COALESCE(s_amount_3x_avg, false) THEN 20 ELSE 0 END
    + CASE WHEN s_new_device                     THEN 25 ELSE 0 END
    + CASE WHEN s_new_country                    THEN 20 ELSE 0 END
    + CASE WHEN s_5_tx_in_10_min                 THEN 15 ELSE 0 END
    + CASE WHEN s_risky_category                 THEN 10 ELSE 0 END
    + CASE WHEN COALESCE(s_fraud_history, false) THEN 15 ELSE 0 END
    + CASE WHEN s_night_hour                     THEN 10 ELSE 0 END;

UPDATE fact_transaction_risk
SET risk_level_v2 = CASE WHEN risk_score_v2 >= 80 THEN 'CRITICAL'
                         WHEN risk_score_v2 >= 60 THEN 'HIGH'
                         WHEN risk_score_v2 >= 30 THEN 'MEDIUM'
                         ELSE 'LOW' END;
```

Microsoft SQL Server (SSMS)

```
IF COL_LENGTH('fact_transaction_risk', 'risk_score_v2') IS NULL
    ALTER TABLE fact_transaction_risk ADD risk_score_v2 INT;
IF COL_LENGTH('fact_transaction_risk', 'risk_level_v2') IS NULL
    ALTER TABLE fact_transaction_risk ADD risk_level_v2 VARCHAR(10);
GO   -- new columns must exist before the next batch uses them

UPDATE fact_transaction_risk
SET risk_score_v2 =
      CASE WHEN s_amount_3x_avg = 1  THEN 20 ELSE 0 END
    + CASE WHEN s_new_device = 1     THEN 25 ELSE 0 END
    + CASE WHEN s_new_country = 1    THEN 20 ELSE 0 END
    + CASE WHEN s_5_tx_in_10_min = 1 THEN 15 ELSE 0 END
    + CASE WHEN s_risky_category = 1 THEN 10 ELSE 0 END
    + CASE WHEN s_fraud_history = 1  THEN 15 ELSE 0 END
    + CASE WHEN s_night_hour = 1     THEN 10 ELSE 0 END;

UPDATE fact_transaction_risk
SET risk_level_v2 = CASE WHEN risk_score_v2 >= 80 THEN 'CRITICAL'
                         WHEN risk_score_v2 >= 60 THEN 'HIGH'
                         WHEN risk_score_v2 >= 30 THEN 'MEDIUM'
                         ELSE 'LOW' END;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`ADD COLUMN IF NOT EXISTS`	`IF COL_LENGTH('tabela', 'kolumna') IS NULL ALTER TABLE ... ADD ...`	SQL Server nie zna IF NOT EXISTS przy kolumnach ani słowa COLUMN.
`TEXT`	`VARCHAR(10)`	TEXT jest w SQL Server przestarzały.
`(brak)`	`GO`	Bez GO SSMS zgłosi „Invalid column name 'risk_score_v2'", bo sprawdza nazwy przed dodaniem kolumny.
`CASE WHEN COALESCE(s, false)`	`CASE WHEN s = 1`	Warunek na BIT; NULL-i nie ma (patrz 03).

Funkcje i metody użyte w tym zapytaniu

- **ALTER TABLE ... ADD COLUMN IF NOT EXISTS**: Dodaje nową kolumnę do tabeli; IF NOT EXISTS pomija ten krok, gdy kolumna już jest (skrypt można puścić drugi raz). SQL Server nie zna IF NOT EXISTS ani słowa COLUMN: sprawdzamy istnienie funkcją COL_LENGTH('tabela', 'kolumna'), która zwraca NULL, gdy kolumny nie ma.
- **TEXT**: W PostgreSQL TEXT to zwykły tekst bez limitu długości. W SQL Server typ TEXT jest przestarzały, więc używamy VARCHAR(n) lub VARCHAR(MAX).

- **GO (tylko SSMS)**: Separator paczek (batchy) w SSMS, nie jest poleceniem SQL. Potrzebny, gdy w jednej paczce dodajemy kolumnę i od razu jej używamy: SQL Server sprawdza nazwy kolumn przed wykonaniem całej paczki i zgłosiłby błąd „Invalid column name". W PostgreSQL nie ma takiej potrzeby.
- **UPDATE ... SET**: Zmienia wartości w istniejących wierszach. Bez WHERE zmienia wszystkie wiersze tabeli.
- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli": dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **COALESCE(x, y) / ISNULL(x, y)**: Zwraca pierwszą niepustą wartość, np. 0 zamiast NULL. COALESCE działa w obu systemach; ISNULL to wersja tylko z SQL Server (dokładnie 2 argumenty).
- **BOOLEAN vs BIT**: PostgreSQL ma prawdziwy typ logiczny BOOLEAN z wartościami true / false. SQL Server go nie ma: używa BIT z wartościami 1 / 0. Dlatego w T-SQL zawsze piszemy porównanie, np. is_fraud = 1, a w PostgreSQL wystarczy samo is_fraud.

2. Część 2: wynik v2 klienta (nowe kolumny w fact_customer_risk)

Co robi: To samo co część 2 skryptu 03, ale z wynikiem v2 i do nowych kolumn risk_score_v2 / risk_level_v2. Kolumny v1 zostają, więc w tabeli klienta są obie wersje obok siebie.

Wynik na Twoich danych: Klienci v2: LOW 8 168, MEDIUM 1 740, HIGH 68, CRITICAL 24.

PostgreSQL (Twój skrypt)

```
ALTER TABLE fact_customer_risk
    ADD COLUMN IF NOT EXISTS risk_score_v2 NUMERIC(5,2),
    ADD COLUMN IF NOT EXISTS risk_level_v2 VARCHAR(10);

UPDATE fact_customer_risk r
SET risk_score_v2 = COALESCE(c.max_score, 0),
    risk_level_v2 = CASE WHEN COALESCE(c.max_score, 0) >= 80 THEN 'CRITICAL'
                         WHEN COALESCE(c.max_score, 0) >= 60 THEN 'HIGH'
                         WHEN COALESCE(c.max_score, 0) >= 30 THEN 'MEDIUM'
                         ELSE 'LOW' END
FROM (
    SELECT cu.customer_id, MAX(tr.risk_score_v2) AS max_score
    FROM dim_customer cu
    LEFT JOIN fact_transaction_risk tr
           ON tr.customer_id = cu.customer_id
          AND tr.tx_datetime >= TIMESTAMP '2026-07-01' - INTERVAL '30 days'
    GROUP BY cu.customer_id
) c
WHERE c.customer_id = r.customer_id;
```

Microsoft SQL Server (SSMS)

```
IF COL_LENGTH('fact_customer_risk', 'risk_score_v2') IS NULL
    ALTER TABLE fact_customer_risk ADD risk_score_v2 NUMERIC(5,2);
IF COL_LENGTH('fact_customer_risk', 'risk_level_v2') IS NULL
    ALTER TABLE fact_customer_risk ADD risk_level_v2 VARCHAR(10);
GO

UPDATE r
SET risk_score_v2 = COALESCE(c.max_score, 0),
    risk_level_v2 = CASE WHEN COALESCE(c.max_score, 0) >= 80 THEN 'CRITICAL'
                         WHEN COALESCE(c.max_score, 0) >= 60 THEN 'HIGH'
                         WHEN COALESCE(c.max_score, 0) >= 30 THEN 'MEDIUM'
                         ELSE 'LOW' END
FROM fact_customer_risk r
JOIN (
    SELECT cu.customer_id, MAX(tr.risk_score_v2) AS max_score
    FROM dim_customer cu
    LEFT JOIN fact_transaction_risk tr
           ON tr.customer_id = cu.customer_id
          AND tr.tx_datetime >= DATEADD(day, -30, CAST('2026-07-01' AS datetime2))
    GROUP BY cu.customer_id
) c ON c.customer_id = r.customer_id;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`ADD COLUMN IF NOT EXISTS`	`IF COL_LENGTH(...) IS NULL ... ADD + GO`	Jak w części 1.
`UPDATE ... FROM (...) c WHERE`	`UPDATE r ... FROM tabela r JOIN (...) c ON`	Składnia aktualizacji z innej tabeli.
`TIMESTAMP ... - INTERVAL '30 days'`	`DATEADD(day, -30, ...)`	Data początku okna 30 dni.

Funkcje i metody użyte w tym zapytaniu

- **ALTER TABLE ... ADD COLUMN IF NOT EXISTS**: Dodaje nową kolumnę do tabeli; IF NOT EXISTS pomija ten krok, gdy kolumna już jest (skrypt można puścić drugi raz). SQL Server nie zna IF NOT EXISTS ani słowa COLUMN: sprawdzamy istnienie funkcją COL_LENGTH('tabela', 'kolumna'), która zwraca NULL, gdy kolumny nie ma.
- **GO (tylko SSMS)**: Separator paczek (batchy) w SSMS, nie jest poleceniem SQL. Potrzebny, gdy w jednej paczce dodajemy kolumnę i od razu jej używamy: SQL Server sprawdza nazwy kolumn przed wykonaniem całej paczki i zgłosiłby błąd „Invalid column name". W PostgreSQL nie ma takiej potrzeby.
- **UPDATE ... SET**: Zmienia wartości w istniejących wierszach. Bez WHERE zmienia wszystkie wiersze tabeli.
- **UPDATE ... FROM (aktualizacja z innej tabeli)**: Aktualizuje tabelę wartościami policzonymi w innym zapytaniu. PostgreSQL: UPDATE tabela alias SET ... FROM (podzapytanie) c WHERE c.id = alias.id. SQL Server: UPDATE alias SET ... FROM tabela alias JOIN (podzapytanie) c ON c.id = alias.id, czyli warunek łączenia jest w JOIN, a po UPDATE stoi alias.
- **COALESCE(x, y) / ISNULL(x, y)**: Zwraca pierwszą niepustą wartość, np. 0 zamiast NULL. COALESCE działa w obu systemach; ISNULL to wersja tylko z SQL Server (dokładnie 2 argumenty).
- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli": dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **MIN(x) / MAX(x)**: Najmniejsza / największa wartość w grupie (np. najwyższy wynik ryzyka klienta).
- **LEFT JOIN**: Jak JOIN, ale zostawia wszystkie wiersze z lewej tabeli; tam, gdzie nie ma pary, kolumny z prawej tabeli są puste (NULL). Używamy np. do znalezienia transakcji bez kursu waluty.
- **Stała data: DATE '2026-01-01'**: W PostgreSQL DATE '...' i TIMESTAMP '...' tworzą wartość daty z tekstu. W SQL Server wystarczy tekst '2026-01-01' (porównywany z kolumną daty) albo CAST('2026-01-01' AS date).
- **INTERVAL '24 hours' vs DATEADD**: PostgreSQL ma typ „odcinek czasu": ts + INTERVAL '24 hours' to data przesunięta o dobę. SQL Server nie ma takiego typu; używamy DATEADD(hour, 24, ts) (albo minute, day).
- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.

3. Porównanie 3a: progi, v1 vs v2 obok siebie

Co robi: Krok scores „rozkłada" każdą transakcję na dwa wiersze: jeden z wynikiem v1, drugi z wynikiem v2 (CROSS JOIN LATERAL z VALUES). Potem, jak w 04, każdy wiersz jest sprawdzany przy 5 progach, liczymy macierz pomyłek osobno dla każdej pary próg + wersja i wskaźniki precision, recall i FPR. Czytamy parami wierszy: ten sam próg, dwie wersje.

Wynik na Twoich danych: Próg 50: v1 łapie 2 273 fraudy przy 4 307 fałszywych alarmach, v2 łapie 2 834 przy 1 452. Próg 80 w v2: 466 alarmów, 99,1% trafnych.

PostgreSQL (Twój skrypt)

```
WITH scores AS (
    SELECT v.version, v.score, r.is_fraud
    FROM fact_transaction_risk r
    CROSS JOIN LATERAL (VALUES ('v1 (plan)', r.risk_score),          -- one row per version (T-SQL: CROSS APPLY)
                               ('v2 (tuned)', r.risk_score_v2)) AS v(version, score)
),
matrix AS (
    SELECT
        th.threshold, s.version,
        COUNT(*) FILTER (WHERE s.score >= th.threshold)                    AS flagged,
        COUNT(*) FILTER (WHERE s.score >= th.threshold AND s.is_fraud)     AS tp,
        COUNT(*) FILTER (WHERE s.score >= th.threshold AND NOT s.is_fraud) AS fp,
        COUNT(*) FILTER (WHERE s.score <  th.threshold AND s.is_fraud)     AS fn,
        COUNT(*) FILTER (WHERE s.score <  th.threshold AND NOT s.is_fraud) AS tn
    FROM (VALUES (30), (50), (60), (70), (80)) AS th(threshold)
    CROSS JOIN scores s
    GROUP BY th.threshold, s.version
)
SELECT
    threshold,
    version,
    flagged,
    tp AS fraud_detected,
    fp AS false_alarms,
    ROUND(100.0 * tp / NULLIF(tp + fp, 0), 1)           AS precision_pct,
    ROUND(100.0 * tp / NULLIF(tp + fn, 0), 1)           AS recall_pct,
    ROUND(100.0 * fp / NULLIF(fp + tn, 0), 2)           AS false_positive_rate_pct
FROM matrix
ORDER BY threshold, version;
```

Microsoft SQL Server (SSMS)

```
WITH scores AS (
    SELECT v.version, v.score, r.is_fraud
    FROM fact_transaction_risk r
    CROSS APPLY (VALUES ('v1 (plan)', CAST(r.risk_score AS int)),
```

```
                      ('v2 (tuned)', r.risk_score_v2)) AS v(version, score)
),
matrix AS (
    SELECT
        th.threshold, s.version,
        SUM(CASE WHEN s.score >= th.threshold THEN 1 ELSE 0 END)                    AS flagged,
        SUM(CASE WHEN s.score >= th.threshold AND s.is_fraud = 1 THEN 1 ELSE 0 END)  AS tp,
        SUM(CASE WHEN s.score >= th.threshold AND s.is_fraud = 0 THEN 1 ELSE 0 END)  AS fp,
        SUM(CASE WHEN s.score <  th.threshold AND s.is_fraud = 1 THEN 1 ELSE 0 END)  AS fn,
        SUM(CASE WHEN s.score <  th.threshold AND s.is_fraud = 0 THEN 1 ELSE 0 END)  AS tn
    FROM (VALUES (30), (50), (60), (70), (80)) AS th(threshold)
    CROSS JOIN scores s
    GROUP BY th.threshold, s.version
)
SELECT
    threshold,
    version,
    flagged,
    tp AS fraud_detected,
    fp AS false_alarms,
    ROUND(100.0 * tp / NULLIF(tp + fp, 0), 1)           AS precision_pct,
    ROUND(100.0 * tp / NULLIF(tp + fn, 0), 1)           AS recall_pct,
    ROUND(100.0 * fp / NULLIF(fp + tn, 0), 2)           AS false_positive_rate_pct
FROM matrix
ORDER BY threshold, version;
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`CROSS JOIN LATERAL (VALUES ...)`	`CROSS APPLY (VALUES ...)`	Jeden wiersz na wersję wyniku.
`r.risk_score`	`CAST(r.risk_score AS int)`	Zabezpieczenie: obie wartości w jednej kolumnie VALUES powinny mieć ten sam typ (wynik jest zawsze liczbą całkowitą).
`COUNT(*) FILTER (...)`	`SUM(CASE ... END)`	Liczenie warunkowe.

Funkcje i metody użyte w tym zapytaniu

- **WITH nazwa AS (...) (CTE)**: „Tymczasowa tabela na czas jednego zapytania”. Dzielimy duże zapytanie na nazwane kroki, a każdy kolejny krok może korzystać z poprzednich. Działa tak samo w obu systemach.
- **CROSS JOIN LATERAL (VALUES ...) vs CROSS APPLY (VALUES ...)**: „Obraca” kolumny w wiersze (unpivot): z jednego wiersza z 7 kolumnami sygnałów robi 7 wierszy (nazwa sygnału, czy włączony). LATERAL pozwala małej tabeli VALUES korzystać z kolumn bieżącego wiersza. W SQL Server to samo robi CROSS APPLY.
- **(VALUES (30), (50), ...) AS th(threshold)**: Mała tabela wpisana ręcznie w zapytaniu, tu lista progów. AS th(threshold) nadaje jej nazwę i nazwę kolumny. Działa tak samo w obu systemach.
- **CROSS JOIN**: Łączy każdy wiersz z każdym (iloczyn kartezjański): 5 progów × 257 947 transakcji = każda transakcja sprawdzona przy każdym progu.
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AND / OR / NOT**: Łączenie warunków: oba naraz (AND), choć jeden (OR), zaprzeczenie (NOT). W PostgreSQL NOT is_fraud działa na kolumnie BOOLEAN; w T-SQL piszemy is_fraud = 0.
- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **100.0 * a / b (dzielenie z ułamkiem)**: Dzielenie dwóch liczb całkowitych daje w obu systemach wynik całkowity (7/2 = 3). Mnożenie przez 100.0 lub 1.0 zamienia liczbę na dziesiętną i wynik ma ułamek.
- **NULLIF(x, 0)**: Zwraca NULL, gdy x = 0. Dzielenie przez NULL daje NULL zamiast błędu „dzielenie przez zero”. Tak samo w obu systemach.
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

4. Porównanie 3b: transakcje według poziomu, v1 vs v2

Co robi: Każda transakcja dostaje dwa wiersze (poziom v1 i poziom v2). Grupujemy po wersji i poziomie i liczymy liczbę transakcji, fraudów i fraud rate. ORDER BY z CASE ustawia poziomy w kolejności LOW → CRITICAL (alfabetycznie byłoby inaczej). Dobry scoring to „strome schody": im wyższy poziom, tym dużo wyższy fraud rate.

Wynik na Twoich danych: v2: LOW 0,20%, MEDIUM 12,75%, HIGH 85,16%, CRITICAL 99,14% (v1: 0,26 / 5,32 / 56,00 / 90,39%).

PostgreSQL (Twój skrypt)

```
SELECT
    v.version,
    v.risk_level,
    COUNT(*)                                          AS transactions,
    COUNT(*) FILTER (WHERE r.is_fraud)                AS fraud_transactions,
    ROUND(100.0 * AVG(r.is_fraud::int), 2)            AS fraud_rate_pct
FROM fact_transaction_risk r
CROSS JOIN LATERAL (VALUES ('v1 (plan)', r.risk_level),
                           ('v2 (tuned)', r.risk_level_v2)) AS v(version, risk_level)
GROUP BY v.version, v.risk_level
ORDER BY v.version, MIN(CASE v.risk_level WHEN 'LOW' THEN 1 WHEN 'MEDIUM' THEN 2 WHEN 'HIGH' THEN 3 ELSE 4 END);
```

Microsoft SQL Server (SSMS)

```
SELECT
    v.version,
    v.risk_level,
    COUNT(*)                                          AS transactions,
    SUM(CASE WHEN r.is_fraud = 1 THEN 1 ELSE 0 END)   AS fraud_transactions,
    ROUND(100.0 * AVG(CAST(r.is_fraud AS float)), 2)  AS fraud_rate_pct
FROM fact_transaction_risk r
CROSS APPLY (VALUES ('v1 (plan)', r.risk_level),
                    ('v2 (tuned)', r.risk_level_v2)) AS v(version, risk_level)
GROUP BY v.version, v.risk_level
ORDER BY v.version, MIN(CASE v.risk_level WHEN 'LOW' THEN 1 WHEN 'MEDIUM' THEN 2 WHEN 'HIGH' THEN 3 ELSE 4 END);
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`CROSS JOIN LATERAL`	`CROSS APPLY`	Unpivot wersji.
`FILTER / AVG(::int)`	`SUM(CASE ...) / AVG(CAST AS float)`	Liczenie fraudów.

Funkcje i metody użyte w tym zapytaniu

- **CROSS JOIN LATERAL (VALUES ...) vs CROSS APPLY (VALUES ...)**: „Obraca" kolumny w wiersze (unpivot): z jednego wiersza z 7 kolumnami sygnałów robi 7 wierszy (nazwa sygnału, czy włączony). LATERAL pozwala małej tabeli VALUES korzystać z kolumn bieżącego wiersza. W SQL Server to samo robi CROSS APPLY.
- **(VALUES (30), (50), ...) AS th(threshold)**: Mała tabela wpisana ręcznie w zapytaniu, tu lista progów. AS th(threshold) nadaje jej nazwę i nazwę kolumny. Działa tak samo w obu systemach.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **AVG(is_fraud::int) vs AVG(CAST(is_fraud AS float))**: Średnia z 0/1 to udział jedynek, czyli fraud rate. W PostgreSQL zamieniamy BOOLEAN na liczbę (::int). W SQL Server BIT trzeba zamienić na float, bo AVG z liczby całkowitej dałoby wynik całkowity (0).
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli": dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **MIN(x) / MAX(x)**: Najmniejsza / największa wartość w grupie (np. najwyższy wynik ryzyka klienta).
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

5. Porównanie 3c: klienci według poziomu, v1 vs v2

Co robi: To samo co 3b, ale dla klientów (fact_customer_risk, stan na 30.06.2026): ilu klientów jest na każdym poziomie w v1 i w v2.

Wynik na Twoich danych: v1: 7 734 / 2 066 / 143 / 57; v2: 8 168 / 1 740 / 68 / 24 (LOW / MEDIUM / HIGH / CRITICAL).

PostgreSQL (Twój skrypt)

```
SELECT
    v.version,
    v.risk_level,
    COUNT(*)                                            AS customers
FROM fact_customer_risk r
CROSS JOIN LATERAL (VALUES ('v1 (plan)', r.risk_level),
                           ('v2 (tuned)', r.risk_level_v2)) AS v(version, risk_level)
GROUP BY v.version, v.risk_level
ORDER BY v.version, MIN(CASE v.risk_level WHEN 'LOW' THEN 1 WHEN 'MEDIUM' THEN 2 WHEN 'HIGH' THEN 3 ELSE 4 END);
```

Microsoft SQL Server (SSMS)

```
SELECT
    v.version,
    v.risk_level,
    COUNT(*)                                            AS customers
FROM fact_customer_risk r
CROSS APPLY (VALUES ('v1 (plan)', r.risk_level),
                    ('v2 (tuned)', r.risk_level_v2)) AS v(version, risk_level)
GROUP BY v.version, v.risk_level
ORDER BY v.version, MIN(CASE v.risk_level WHEN 'LOW' THEN 1 WHEN 'MEDIUM' THEN 2 WHEN 'HIGH' THEN 3 ELSE 4 END);
```

Różnice

PostgreSQL	SQL Server	Dlaczego
`CROSS JOIN LATERAL`	`CROSS APPLY`	Jedyna różnica.

Funkcje i metody użyte w tym zapytaniu

- **CROSS JOIN LATERAL (VALUES ...) vs CROSS APPLY (VALUES ...)**: „Obraca” kolumny w wiersze (unpivot): z jednego wiersza z 7 kolumnami sygnałów robi 7 wierszy (nazwa sygnału, czy włączony). LATERAL pozwala małej tabeli VALUES korzystać z kolumn bieżącego wiersza. W SQL Server to samo robi CROSS APPLY.
- **(VALUES (30), (50), ...) AS th(threshold)**: Mała tabela wpisana ręcznie w zapytaniu, tu lista progów. AS th(threshold) nadaje jej nazwę i nazwę kolumny. Działa tak samo w obu systemach.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli”: dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **MIN(x) / MAX(x)**: Najmniejsza / największa wartość w grupie (np. najwyższy wynik ryzyka klienta).
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).

Słowniczek: wszystkie funkcje i metody

Zebrane w jednym miejscu, alfabetycznie.

- **(VALUES (30), (50), ...) AS th(threshold)**: Mała tabela wpisana ręcznie w zapytaniu, tu lista progów. AS th(threshold) nadaje jej nazwę i nazwę kolumny. Działa tak samo w obu systemach.
- **100.0 * a / b (dzielenie z ułamkiem)**: Dzielenie dwóch liczb całkowitych daje w obu systemach wynik całkowity (7/2 = 3). Mnożenie przez 100.0 lub 1.0 zamienia liczbę na dziesiętną i wynik ma ułamek.
- **::date vs CAST(... AS date)**: Zamiana typu. W PostgreSQL skrót x::date (tu: obcina godzinę z daty-czasu). W SQL Server CAST(x AS date) lub CONVERT(date, x). Zapis CAST działa też w PostgreSQL.
- **::typ vs CAST(... AS typ)**: Zamiana typu, np. ::numeric, ::int, ::text. W SQL Server CAST(x AS typ). Tekstowy odpowiednik ::text to CAST(x AS VARCHAR(10)).
- **ALTER TABLE ... ADD COLUMN IF NOT EXISTS**: Dodaje nową kolumnę do tabeli; IF NOT EXISTS pomija ten krok, gdy kolumna już jest (skrypt można puścić drugi raz). SQL Server nie zna IF NOT EXISTS ani słowa COLUMN: sprawdzamy istnienie funkcją COL_LENGTH('tabela', 'kolumna'), która zwraca NULL, gdy kolumny nie ma.
- **ALTER TABLE ... ADD PRIMARY KEY**: Dodaje klucz główny do istniejącej tabeli. W SQL Server kolumna musi być najpierw oznaczona jako NOT NULL, a klucz zwykle dostaje nazwę (ADD CONSTRAINT nazwa PRIMARY KEY).
- **AND / OR / NOT**: Łączenie warunków: oba naraz (AND), choć jeden (OR), zaprzeczenie (NOT). W PostgreSQL NOT is_fraud działa na kolumnie BOOLEAN; w T-SQL piszemy is_fraud = 0.
- **AS (alias)**: Nadaje kolumnie lub tabeli krótką nazwę. t.customer_id znaczy: kolumna customer_id z tabeli o aliasie t.
- **AVG(is_fraud::int) vs AVG(CAST(is_fraud AS float))**: Średnia z 0/1 to udział jedynek, czyli fraud rate. W PostgreSQL zamieniamy BOOLEAN na liczbę (::int). W SQL Server BIT trzeba zamienić na float, bo AVG z liczby całkowitej dałoby wynik całkowity (0).
- **AVG(x)**: Średnia arytmetyczna. Uwaga w SQL Server: AVG z kolumny całkowitej (INT, wynik COUNT) zwraca liczbę całkowitą z obciętym ułamkiem (np. 25 zamiast 25,8). Dlatego w T-SQL mnożymy przez 1.0 albo rzutujemy na float.
- **BETWEEN a AND b / NOT BETWEEN**: Czy wartość leży w przedziale, włącznie z końcami. NOT BETWEEN odwraca warunek.
- **BOOL_OR(x) vs MAX(CAST(x AS int))**: Czy choć jedna wartość w grupie jest prawdą (np. czy klient miał choć jeden fraud). SQL Server nie ma BOOL_OR; MAX z 0/1 daje ten sam efekt (1 = tak).
- **BOOLEAN vs BIT**: PostgreSQL ma prawdziwy typ logiczny BOOLEAN z wartościami true / false. SQL Server go nie ma: używa BIT z wartościami 1 / 0. Dlatego w T-SQL zawsze piszemy porównanie, np. is_fraud = 1, a w PostgreSQL wystarczy samo is_fraud.
- **CASE WHEN ... THEN ... ELSE ... END**: Instrukcja „jeżeli”: dla każdego wiersza sprawdza warunki po kolei i zwraca wartość z pierwszego spełnionego. Działa tak samo w obu systemach (różnica jest tylko w warunkach na typie logicznym).
- **CHECK (warunek)**: Reguła sprawdzana przy każdym zapisie, np. wiek od 18 do 100. Wiersz, który jej nie spełnia, zostanie odrzucony.
- **COALESCE(x, y) / ISNULL(x, y)**: Zwraca pierwszą niepustą wartość, np. 0 zamiast NULL. COALESCE działa w obu systemach; ISNULL to wersja tylko z SQL Server (dokładnie 2 argumenty).
- **CONCAT_WS(' + ', a, b, ...)**: Skleja teksty z separatorem i pomija puste (NULL). Tu z nazw włączonych sygnałów robi napis typu „amount 3x + risky category + history”. W SQL Server od wersji 2017.
- **COUNT(*)**: Liczy wiersze w grupie (albo w całej tabeli, gdy nie ma GROUP BY).
- **COUNT(*) FILTER (WHERE ...) vs SUM(CASE ...)**: Liczy tylko wiersze spełniające warunek, np. same fraudy. To funkcja PostgreSQL. W SQL Server zastępujemy ją SUM(CASE WHEN warunek THEN 1 ELSE 0 END) albo COUNT(CASE WHEN warunek THEN 1 END), a SUM(x) FILTER (...) zapisujemy jako SUM(CASE WHEN warunek THEN x END).
- **COUNT(DISTINCT x)**: Liczy różne wartości, np. ile różnych urządzeń albo dni. COUNT(*) − COUNT(DISTINCT id) pokazuje liczbę duplikatów.
- **CREATE INDEX**: Indeks to „spis treści” dla kolumn: przyspiesza wyszukiwanie wierszy (tu: transakcji danego klienta w danym czasie).
- **CREATE TABLE**: Tworzy nową, pustą tabelę: nazwy kolumn, ich typy i reguły. Działa tak samo w obu systemach.
- **CREATE TABLE ... AS SELECT vs SELECT ... INTO**: Tworzy nową tabelę od razu wypełnioną wynikiem zapytania; kolumny i typy biorą się z SELECT-a. W PostgreSQL piszemy CREATE TABLE nazwa AS SELECT ..., a w SQL Server SELECT ... INTO nazwa FROM ... (słowo INTO stoi po liście kolumn, przed FROM).
- **CROSS JOIN**: Łączy każdy wiersz z każdym (iloczyn kartezjański): 5 progów × 257 947 transakcji = każda transakcja sprawdzona przy każdym progu.
- **CROSS JOIN LATERAL (VALUES ...) vs CROSS APPLY (VALUES ...)**: „Obraca” kolumny w wiersze (unpivot): z jednego wiersza z 7 kolumnami sygnałów robi 7 wierszy (nazwa sygnału, czy włączony). LATERAL pozwala małej tabeli VALUES korzystać z kolumn bieżącego wiersza. W SQL Server to samo robi CROSS APPLY.
- **DATE_TRUNC('month' / 'week', ts)**: Obcina datę do początku miesiąca lub tygodnia (tydzień od poniedziałku). SQL Server 2022: DATETRUNC(month, ts) / DATETRUNC(iso_week, ts). Starsze wersje: DATEFROMPARTS(YEAR(ts), MONTH(ts), 1) dla miesiąca.

- **DATEPART(weekday, ts) i @@DATEFIRST**: DATEPART(weekday) zwraca numer dnia tygodnia, ale zależny od ustawienia @@DATEFIRST (dla USA tydzień zaczyna się w niedzielę). Wzór DATEADD(day, -((DATEPART(weekday, d) + @@DATEFIRST - 2) % 7), d) cofa datę do poniedziałku niezależnie od tego ustawienia.
- **DEFAULT**: Wartość wpisywana automatycznie, gdy przy dodawaniu wiersza nie podamy tej kolumny.
- **DROP TABLE IF EXISTS**: Usuwa tabelę, jeśli istnieje; jeśli nie, nie zgłasza błędu. Dzięki temu skrypt można uruchamiać wiele razy. W SQL Server działa od wersji 2016.
- **EXTRACT(HOUR FROM ts) vs DATEPART(hour, ts)**: Wyciąga część daty, tu godzinę 0–23.
- **Funkcja okna: ... OVER (PARTITION BY ... ORDER BY ...)**: Liczy wartość dla każdego wiersza na podstawie „okna" innych wierszy, ale nie skleja wierszy w grupy (w przeciwieństwie do GROUP BY). PARTITION BY dzieli dane na osobne okna (np. osobno dla każdego klienta), ORDER BY ustala kolejność w oknie. Składnia jest taka sama w obu systemach.
- **GO (tylko SSMS)**: Separator paczek (batchy) w SSMS, nie jest poleceniem SQL. Potrzebny, gdy w jednej paczce dodajemy kolumnę i od razu jej używamy: SQL Server sprawdza nazwy kolumn przed wykonaniem całej paczki i zgłosiłby błąd „Invalid column name". W PostgreSQL nie ma takiej potrzeby.
- **GROUP BY**: Dzieli wiersze na grupy (np. po kategorii) i liczy funkcje agregujące (COUNT, SUM, AVG) osobno dla każdej grupy.
- **GROUP BY 1 (numer kolumny)**: W PostgreSQL można grupować po numerze kolumny z SELECT-a. SQL Server na to nie pozwala: trzeba powtórzyć całe wyrażenie (np. cały CASE) albo policzyć je wcześniej w podzapytaniu / CROSS APPLY i grupować po aliasie.
- **HAVING**: Filtr na grupach, działa po GROUP BY (np. tylko kraje z co najmniej 500 transakcjami). WHERE filtruje wiersze przed grupowaniem, HAVING grupy po.
- **IN ('a', 'b', ...)**: Czy wartość jest jedną z listy; krótszy zapis wielu OR.
- **information_schema.tables**: Systemowy widok ze spisem tabel w bazie. Istnieje w obu systemach. Domyślny schemat (folder na tabele) w PostgreSQL to public, a w SQL Server dbo.
- **INTERVAL '24 hours' vs DATEADD**: PostgreSQL ma typ „odcinek czasu": ts + INTERVAL '24 hours' to data przesunięta o dobę. SQL Server nie ma takiego typu; używamy DATEADD(hour, 24, ts) (albo minute, day).
- **IS NULL / IS NOT NULL**: Sprawdza, czy pole jest puste. Nie można pisać = NULL, bo porównanie z NULL nigdy nie daje „prawdy".
- **JOIN ... ON**: Łączy wiersze z dwóch tabel według warunku, np. transakcja z kursem waluty z tego samego dnia. Zostają tylko wiersze, które mają parę (INNER JOIN).
- **JOIN ... USING (kolumna)**: Skrót w PostgreSQL: gdy obie tabele mają kolumnę o tej samej nazwie, USING (customer_id) znaczy ON a.customer_id = b.customer_id. SQL Server nie zna USING, więc trzeba pisać pełne ON.
- **Kolumna z warunku: (a > b) AS sygnał**: W PostgreSQL wynik porównania jest wartością BOOLEAN i może być kolumną. W SQL Server porównanie nie jest wartością: trzeba napisać CASE WHEN a > b THEN 1 ELSE 0 END (i ewentualnie CAST(... AS bit)).
- **LAG(x)**: Wartość z poprzedniego wiersza w oknie, np. kraj lub urządzenie z poprzedniej transakcji klienta. Pierwsza transakcja dostaje NULL.
- **LEFT JOIN**: Jak JOIN, ale zostawia wszystkie wiersze z lewej tabeli; tam, gdzie nie ma pary, kolumny z prawej tabeli są puste (NULL). Używamy np. do znalezienia transakcji bez kursu waluty.
- **LIMIT n vs TOP n**: Zwraca tylko pierwsze n wierszy (po sortowaniu). PostgreSQL: LIMIT n na końcu zapytania. SQL Server: SELECT TOP n tuż po SELECT.
- **MIN(x) / MAX(x)**: Najmniejsza / największa wartość w grupie (np. najwyższy wynik ryzyka klienta).
- **MIN(x) OVER (PARTITION BY ...)**: Najmniejsza wartość w całym oknie, dopisana do każdego wiersza. Tu: kiedy klient pierwszy raz użył danego urządzenia i kiedy zrobił pierwszą transakcję w ogóle.
- **NOT EXISTS (podzapytanie)**: Prawda, gdy podzapytanie nie zwraca żadnego wiersza. Tu: transakcja, dla której nie ma klienta / sprzedawcy / urządzenia, czyli „osierocony" rekord. Tak samo w obu systemach.
- **NOT NULL**: Kolumna nie może być pusta (NULL = brak wartości).
- **NULLIF(x, 0)**: Zwraca NULL, gdy x = 0. Dzielenie przez NULL daje NULL zamiast błędu „dzielenie przez zero". Tak samo w obu systemach.
- **ORDER BY**: Sortuje wynik; DESC malejąco, domyślnie rosnąco. W obu systemach można sortować po aliasie kolumny (np. ORDER BY fraud_rate_pct) i po numerze kolumny (ORDER BY 1).
- **OUTER APPLY (podzapytanie)**: T-SQL: dla każdego wiersza uruchamia podzapytanie, które może korzystać z jego kolumn (tu: policz transakcje tego klienta z ostatnich 10 minut). OUTER zostawia wiersz, nawet gdy podzapytanie nic nie zwróci. Odpowiednik w PostgreSQL: LEFT JOIN LATERAL.
- **PERCENTILE_CONT(p) WITHIN GROUP (ORDER BY x)**: Percentyl z interpolacją: 0.5 to mediana, 0.999 to wartość, poniżej której jest 99,9% danych. W PostgreSQL działa jak zwykła funkcja agregująca (z GROUP BY). W SQL Server tylko jako funkcja okna: trzeba dopisać OVER (PARTITION BY ...) lub OVER () i usunąć powtórzenia (DISTINCT / TOP 1).
- **Podzapytanie skalarne (SELECT ...)**: Zapytanie w nawiasie, które zwraca jedną liczbę i można go użyć jak zwykłej wartości w kolumnie lub warunku.
- **PRIMARY KEY (a, b)**: Klucz złożony z dwóch kolumn: unikalna musi być dopiero para wartości (np. data + waluta), a nie każda kolumna osobno.

- **PRIMARY KEY (klucz główny)**: Kolumna (lub zestaw kolumn), która jednoznacznie identyfikuje wiersz. Nie może się powtarzać ani być pusta. Baza sama zakłada na niej indeks, więc wyszukiwanie po kluczu jest szybkie.
- **RANGE BETWEEN INTERVAL '10 minutes' PRECEDING AND CURRENT ROW**: Ramka okna liczona w CZASIE, nie w wierszach: wszystkie transakcje klienta z ostatnich 10 minut. To jedna z największych przewag PostgreSQL. SQL Server tego nie obsługuje, więc liczymy to podzapytaniem w OUTER APPLY z warunkiem na czas (BETWEEN DATEADD(minute, -10, ts) AND ts).
- **REFERENCES (klucz obcy, FOREIGN KEY)**: Kolumna wskazuje na klucz główny innej tabeli. Baza nie pozwoli wpisać wartości, której nie ma w tabeli nadrzędnej (np. transakcji klienta, który nie istnieje). Składnia identyczna w obu systemach.
- **ROUND(x, n)**: Zaokrągla do n miejsc po przecinku. W PostgreSQL wersja z n działa tylko na typie numeric, stąd dodatkowe ::numeric; w SQL Server ROUND działa na każdym typie liczbowym.
- **ROW_NUMBER()**: Numeruje wiersze w oknie 1, 2, 3... według ORDER BY. Tu: która to transakcja klienta (1 = pierwsza) albo pierwsza transakcja w danym kraju.
- **ROWS BETWEEN ... PRECEDING AND ...**: Ramka okna liczona w wierszach. UNBOUNDED PRECEDING AND 1 PRECEDING = wszystkie wcześniejsze wiersze bez bieżącego (średnia „przed" transakcją, bez podglądania wyniku). 6 PRECEDING AND CURRENT ROW = 7 ostatnich dni (średnia krocząca). Tak samo w obu systemach.
- **SELECT * / alias.***: Wszystkie kolumny tabeli lub kroku CTE.
- **SELECT ... FROM ... WHERE**: Podstawa: SELECT wybiera kolumny, FROM wskazuje tabelę, WHERE filtruje wiersze przed liczeniem.
- **Stała data: DATE '2026-01-01'**: W PostgreSQL DATE '...' i TIMESTAMP '...' tworzą wartość daty z tekstu. W SQL Server wystarczy tekst '2026-01-01' (porównywany z kolumną daty) albo CAST('2026-01-01' AS date).
- **SUM(...) OVER () na wyniku GROUP BY**: Funkcja okna liczona na już zgrupowanych wierszach: OVER () = cała tabela wyników. Tu: suma fraudów ze wszystkich poziomów, żeby policzyć udział każdego poziomu. Działa w obu systemach.
- **SUM(x)**: Suma wartości w grupie.
- **Tabela tymczasowa #nazwa (T-SQL)**: Tabela, która istnieje tylko do końca sesji w SSMS. Zapisujemy do niej wynik pośredni i zakładamy indeks, żeby kolejne kroki (OUTER APPLY) działały szybko. W PostgreSQL odpowiednikiem jest CREATE TEMP TABLE, ale tam nie było potrzebne, bo okno RANGE robi to samo w jednym przebiegu.
- **TEXT**: W PostgreSQL TEXT to zwykły tekst bez limitu długości. W SQL Server typ TEXT jest przestarzały, więc używamy VARCHAR(n) lub VARCHAR(MAX).
- **TIMESTAMP vs DATETIME2**: W PostgreSQL TIMESTAMP to data z godziną. Uwaga, pułapka: w SQL Server TIMESTAMP oznacza coś zupełnie innego (licznik wersji wiersza, synonim ROWVERSION). Odpowiednikiem jest DATETIME2.
- **ts − LAG(ts) vs DATEDIFF**: W PostgreSQL odjęcie dwóch dat-czasów daje INTERVAL (np. 00:12:30). SQL Server tak nie potrafi: DATEDIFF(minute, a, b) zwraca liczbę pełnych minut między a i b.
- **Typy danych**: INT: liczba całkowita; SMALLINT: mała liczba całkowita; BIGINT: duża liczba całkowita; CHAR(n): tekst o stałej długości; VARCHAR(n): tekst do n znaków; DATE: sama data; NUMERIC(p,s): liczba dokładna z s miejscami po przecinku (dobra do pieniędzy). Te typy są takie same w obu systemach.
- **UNION ALL**: Skleja wyniki kilku zapytań jeden pod drugim (bez usuwania duplikatów). Każde zapytanie musi mieć tyle samo kolumn.
- **UPDATE ... FROM (aktualizacja z innej tabeli)**: Aktualizuje tabelę wartościami policzonymi w innym zapytaniu. PostgreSQL: UPDATE tabela alias SET ... FROM (podzapytanie) c WHERE c.id = alias.id. SQL Server: UPDATE alias SET ... FROM tabela alias JOIN (podzapytanie) c ON c.id = alias.id, czyli warunek łączenia jest w JOIN, a po UPDATE stoi alias.
- **UPDATE ... SET**: Zmienia wartości w istniejących wierszach. Bez WHERE zmienia wszystkie wiersze tabeli.
- **WINDOW w AS (...)**: Nazwana definicja okna, której używamy wiele razy (OVER w). Skraca zapis. W SQL Server dostępna dopiero od wersji 2022; w starszych trzeba powtórzyć OVER (PARTITION BY ... ORDER BY ...) przy każdej funkcji.
- **WITH nazwa AS (...) (CTE)**: „Tymczasowa tabela na czas jednego zapytania". Dzielimy duże zapytanie na nazwane kroki, a każdy kolejny krok może korzystać z poprzednich. Działa tak samo w obu systemach.